

Master thesis

for the award of the academic degree
Master of Science (M. Sc.)

Pichaya Tappayuthpijarn

First name Last name

Biomedical Engineering, 2021, 5055625

Degree program, matriculation, matriculation number

Topic:

**Accelerating dose calculations in carbon ion therapy
using a Wasserstein GAN**

Prof. Dr. Marianne Maktabi

Chairperson of the Master Examination Committee/1st examiner

Dr. Hans Rabus

2nd examiner(s)

23. 02. 2024

Levy on

Declaration of independence

I hereby declare that the thesis was written independently, has not yet been submitted in the same or a similar version as an examination performance in another course of study and that no aids and sources other than the specified aids and sources, including the specified or described software, have been used.

Köthen, 23.02.2024



Place and date

Signature (s) of the student

Acknowledgement

I extend my deepest gratitude to my supervisor, Dr. Hans Rabus, for giving me the opportunity to conduct the Master thesis at the working group 8.01 "Artificial Intelligence and Simulation in Medicine" at Physikalisch-Technische Bundesanstalt. His unwavering support, valuable time, and guidance were crucial to the successful completion of this research study. I am indebted to Prof. Dr. Marianne Maktabi, for her insightful inputs and support during my studies in the Master's program at Hochschule Anhalt. My heartfelt thanks go to Ms. Miriam Schwarze and Mr. Leo Thomas for their invaluable support, time, positive energy and guidance from the start to the end. Special thanks to Mr. Florian Burger in the HPC team for the technical support and prompt responses. I appreciate my family who always trust, support and motivate me throughout the Master's program and while writing this thesis. Finally, my sincere thanks to all who directly or indirectly contributed to the completion of this thesis.

Contents

Declaration of independence.....	i
Acknowledgement	ii
Contents.....	iii
1 Motivation and goal setting.....	1
1.1 Introduction to the Thesis.....	1
1.2 Aim and Objectives	2
1.3 Significance of the Thesis.....	2
1.4 Definitions of Terms and Acronyms.....	3
1.5 Thesis Outline.....	4
2 Theoretical basics/ state of the art	5
2.1 Deposition of Energy.....	5
2.2 Monte Carlo Simulation (MCS)	6
2.3 Carbon ion radiotherapy	8
2.4 Generative Adversarial Networks (GANs).....	10
2.4.1 Related works.....	10
2.4.2 Architecture & Concept	11
2.4.3 Limitations & Challenges.....	13
2.4.4 Improvement	14
2.5 Wasserstein GAN	15
3 Material and Methods	19
3.1 Conception	19
3.2 Hardware & Software	21
3.3 MC Simulation.....	23
3.3.1 Head phantom	23
3.3.2 Water phantom.....	26
3.4 Data Preprocessing	28
3.4.1 Resampling.....	28
3.4.2 Cropping datasets to focus on a specific region of interest (ROI)	29
3.4.3 Shuffle and split	29
3.4.4 Create custom dataset and Dataloader	30
4 Implementation	33
4.1 The generator model	33
4.2 The critic model	35

4.3 Model Training.....	37
4.4 Performance measurement.....	38
4.5 Code Availability.....	40
5 Evaluation	40
5.1 Hyperparameter Tuning.....	40
5.2 Performance	43
5.3 Discussion.....	50
6 Summary	52
Appendix	53
A. Dataset: MC Simulation of a therapeutic carbon ion beam in head	53
B. MC Simulation Configuration.....	55
C. Dose Map Construction	56
List of symbols and acronyms	58
List of figures.....	59
List of tables	60
Bibliography	60

1 Motivation and goal setting

Carbon ion beam therapy represents an advanced and promising approach for the treatment of various cancers, offering high efficacy, precision, and limited side effects, particularly for radioresistant and deep-seated tumours (Tsuji, 2017). As a critical component of this therapy, the fast calculation of dose prediction during treatment planning is imperative. This need arises from the necessity to personalize treatment plans based on each patient's unique tumour characteristics, optimizing the balance between tumour control and minimizing exposure to healthy tissues. Fast dose prediction also facilitates real-time adjustments, reducing treatment time, and ensuring the intended dose is accurately delivered. Overall, carbon ion therapy holds the potential to significantly enhance cancer treatment outcomes, and rapid dose prediction plays a pivotal role in achieving this goal.

1.1 Introduction to the Thesis

Carbon ion beam therapy is a type of radiation therapy that uses beams of carbon ions to treat cancer. Its unique properties make it attractive for cancer treatment allowing for precise targeting of the tumor and effective treatment for tumors that are resistant to other types of radiation (Tsuji, 2017). Dose distribution prediction is currently done in computerized treatment planning systems that simulate the delivery of radiation. Monte Carlo (MC) simulation is used to calculate the dose distribution of radiation within the patient's body (Seco & Verhaegen, 2016; Vassiliev, 2016). By simulating the paths of individual radiation particles (e.g., photons, protons or carbon ions), it can provide detailed and highly accurate information about how the radiation is deposited in the tumor and surrounding healthy tissues.

Machine learning, and more recently, deep learning (DL) employing convolutional networks, has found application in the realm of treatment planning. Various studies, such as those by (Nguyen et al., 2019), (Yoganathan & Zhang, 2019), and (Barragán-Montero et al., 2021),

have showcased the capability to accurately predict treatment doses across different sites, including prostate, head and neck, and esophagus, utilizing diverse network architectures. Recently, deep learning algorithms have been proven for fast and yet accurate prediction of the dose distribution. The improvement in speed of dose calculation can lead to better treatment planning and better outcomes for patients.

1.2 Aim and Objectives

The objective of this master's thesis is to explore innovative approaches applicable to treatment planning in carbon ion radiation therapy. Specifically, the focus is on the development of a Wasserstein Generative Adversarial Network (WGAN) to speed up the calculation of dose maps in treatment plans for carbon ion cancer therapy. This objective will be accomplished by addressing the following key goals:

1. Conduct research on the utilization of deep learning and artificial intelligence in dose calculation for radiation therapy.
2. Analyze state-of-the-art methodologies and apply them to the context of carbon ion radiation therapy.
3. Gather data from Monte Carlo (MC) radiation transport simulation, DICOM images, and preprocess the data for use in the study.
4. Develop the WGAN model and explore various training strategies and network optimization techniques.
5. Test the model and assess its performance in predicting dose maps, utilizing the Mean Square Error (MSE) as a measure of prediction quality.
6. Analyze and interpret the results obtained from the experiments.

1.3 Significance of the Thesis

The thesis provides an opportunity for researchers and students to explore the trending topics of leveraging artificial intelligence in medicine. It demonstrates proof-of-concept application of using machine learning for fast dose calculation in radiation therapy. This

could be the beneficial as we develop and design newest solution and the result makes it compelling for online adaptive workflows in clinical application. In addition, for the targeted patients, the developed solution can be used as a tool to help address some of the problems stated in an earlier section. This initial work aims to demonstrate the network's ability to predict the dose in head anatomies before proceeding to more specific/targeted applications.

1.4 Definitions of Terms and Acronyms

Artificial Intelligence (AI): This is a simulated intelligence exhibited by machines aiming to mimic the cognitive function of human minds, such as learning and problem-solving (Russell & Norvig, 2010).

Machine Learning (ML): Machine learning is a branch of artificial intelligence that allows a system to learn and improve independently without requiring explicit programming (Douglass, 2020).

Neural Networks (NN): In artificial intelligence, neural networks consist of interconnected nodes which form an input layer, one or more hidden layers, and an output layer (Shanmugamani, 2018).

Deep learning (DL): This involves analyzing and using patterns in data to increase a program's understanding. It is a neural network with three or more layers of interconnected nodes, each building upon the previous layer to refine and optimize the prediction or categorization (Shanmugamani, 2018).

Generative Adversarial Network (GAN): A type of artificial NN framework used in ML and DL. It consists of two NN, a generator, and a discriminator, which are trained together (Goodfellow et al., 2014). The generator creates data, such as images, while the discriminator evaluates the generated data against real data. The two networks compete in a "game," with the generator continually improving its ability to generate data that is indistinguishable from real data, while the discriminator becomes better at telling real from

fake. GANs are commonly used for tasks like image generation, data synthesis, and style transfer.

Wasserstein GAN (WGAN): A type of GAN that introduces the Wasserstein distance as the loss function (Arjovsky et al., 2017) , offering more stable training and better gradient flow, which leads to improved generation of data while avoiding known limitations such as mode collapse. It's designed to provide a smoother and more reliable learning process for GANs (Gulrajani et al., 2017).

Monte Carlo (MC): The Monte Carlo method is a versatile and powerful computational technique widely employed in various domains such as physics, engineering, finance, and statistics (Vassiliev, 2016). This computational technique is used to model and simulate the behavior of radiation particles as they interact with tissues and other materials within the human body. This approach is critical in the field of radiation therapy to accurately calculate and predict the dose distribution of radiation delivered to a patient during treatment.

1.5 Thesis Outline

The rest of this thesis is structured as follows: Chapter two (2) focuses on various state-of-the-art technologies and research related to accomplishing the goals of this thesis. It provides an overview of essential theoretical and practical basics regarding energy deposition in radiation, Generative Adversarial Network (GAN), and Wasserstein GAN as its improvement. Chapter three (3) focuses on the methodology behind the data simulation and data preprocessing. The used system's hardware, software specifications and code are also described for reproducibility. Chapters four (4) concentrates on the software application's implementation detailing the design, functionality, and training strategies of both the generator and critic models. Chapter five (5) concludes the research journey by thoroughly evaluating outcomes. Performance metrics are closely examined. The results and evaluations from the testing will also be presented. Chapter six (6), the last chapter of this thesis, provides the conclusion and outlines future work related to this thesis.

2 Theoretical basics/ state of the art

2.1 Deposition of Energy

The impact of radiation on substances is determined by the radiation field, as defined by radiometric parameters. Radiation engages with matter through a sequence of interactions where the energy of particles undergoes conversion and is ultimately imparted to the material. The dosimetric parameters characterizing these interactions are outlined in the subsequent section, explaining the conversion and deposition of energy. All definitions in this section are taken from (Seltzer et al., 2011)

- **Energy Deposit**

Energy deposition is the process of ionizing radiation losing energy as it interacts with matter. It involves the transfer of energy to atomic or molecular electrons, leading to ionization and excitation of the material. This term is widely used in the field of radiation physics and radiation transport. It is essential for understanding how radiation interacts with and deposits energy in materials, tissues, or biological systems. Energy deposition is fundamental in radiation therapy, radiology, and nuclear physics, as it helps determine the distribution of energy in tissues or materials. This information is crucial for treatment planning and predicting the effects of radiation.

The energy deposit, ε_i , is the energy deposited in a single interaction, i ,

$$\varepsilon_i = \varepsilon_{in} + \varepsilon_{out} + Q \quad \text{Eq. 1}$$

where ε_{in} is the energy of the incident ionizing particle (excluding rest energy), ε_{out} is the sum of the energies of all charged and uncharged ionizing particles leaving the interaction (excluding rest energy), and Q is the change in the rest energies of the nucleus and of all elementary particles involved in the interaction. The unit is J or can be expressed as eV.

- **Energy Imparted**

The energy imparted, ε , to the matter in a given volume is the sum of all energy deposits in the volume, thus

$$\varepsilon = \sum \varepsilon_i \quad \text{Eq. 2}$$

where the summation is performed over all energy deposits, ε_i , in that volume. The unit is J.

- **Absorbed Dose**

The absorbed dose, D , is the quotient of the mean energy imparted by ionizing radiation to matter of mass dm , thus

$$D = \frac{d\bar{\varepsilon}}{dm} \quad \text{Eq. 3}$$

Unit is J/kg or gray (Gy).

2.2 Monte Carlo Simulation (MCS)

The Monte Carlo simulation (MCS) has found extensive application in generating the majority of data in the field of radiation dosimetry by employing stochastic mathematical techniques to calculate approximations of numerical values (Andreo et al., 2017). MCS plays a pivotal role in simulating the physics of radiation transport, offering a robust computational approach. The main concept of the Monte Carlo method is random sampling, where random numbers or probability distributions represent variables associated with the behavior of radiation particles. These random variables are integrated into a simulation model, designed to emulate the complex movement and interaction of radiation in various materials. Before simulation, the problem is mathematically formulated, encompassing these random variables, which may include factors like scattering probabilities and particle energies. Repeated trials, or iterations, are executed, each involving numerous simulations with different initial conditions. These trials generate a wealth of data that allow for the estimation of radiation transport characteristics such as dose distributions. Importantly, the

statistical accuracy of these estimates improves as the number of trials increases, aligning with the law of large numbers. MCS adaptability in radiation transport is crucial in radiotherapy treatment planning.

Treatment planning for radiotherapy of cancer can benefit from MCS as it is a very complex computational problem in modern medical physics. The calculation of three-dimensional (3D) dose distributions in a volume requires approximately 2-3 mm spatial resolution. According to the American Association of Physicists in Medicine (AAPM), the accuracy of patient dose calculations error should be less than 2%. However, the general principle of 3% accuracy in dose delivery is reasonable and realistic (AAPM Report 85, 2004).

MCS had gained popularity for over 30 years since the 1970's. The number of scientific publications related to MCS in medical physics had increased and reached more than 1000 per year in 2010 (Seco & Verhaegen, 2016). However, it is well known that MSC requires long processing time which makes it difficult in some routine clinical practices, where the calculation of patient dose distribution should not take longer than a few minutes. In some use cases, many iterations of recalculation must be performed to achieve the optimal 3D dose distribution. Or in some case that the location of the tumors cannot be immobilized, such as for example with lung tumors, the range of movement extends to over 1 cm. It is usual to take many snapshots of the 3D anatomy at different stage of the respiration and perform several dose calculations separately. Then the calculations are combined to calculate the total delivered dose to the patient (Vassiliev, 2016).

There have been works to explore or propose the development of a framework to utilize Monte Carlo simulation (MCS) on graphics processing units (GPU) as an alternative for high computational power (Bert et al., 2013). This potentially offered a more efficient solution for computationally intensive tasks.

2.3 Carbon ion radiotherapy

Since the late 20th century, charged particle therapy has become popular and gained interest around the world. Charged particle therapy offers several advantages, including its ability to deliver highly targeted radiation doses to cancerous tissue while minimizing damage to surrounding healthy cells. Among protons and other types of ion beam therapy, carbon-ion radiotherapy stands out for its radio physical and biological advantages. Being highly effective, it is suitable for certain types of tumors that are resistant to conventional radiation therapy. The National Institute of Radiological Sciences in Japan was the first place to start utilizing carbon ion beams for cancer treatment. As of October 2023, there are over 14 facilities for carbon-ion radiotherapy in operation, 4 facilities are under construction and even a few are in planning stage (Jermann, 2016). According to the Particle Therapy Co-Operative Group (PTCOG) patient statistics, a total of 362,546 patients were treated with charged particles therapy from 1954-2022. From 1994, there were 312,087 (86.1%) patients treated with proton beam and 46,872 (12.9%) patients treated with carbon ion beam.

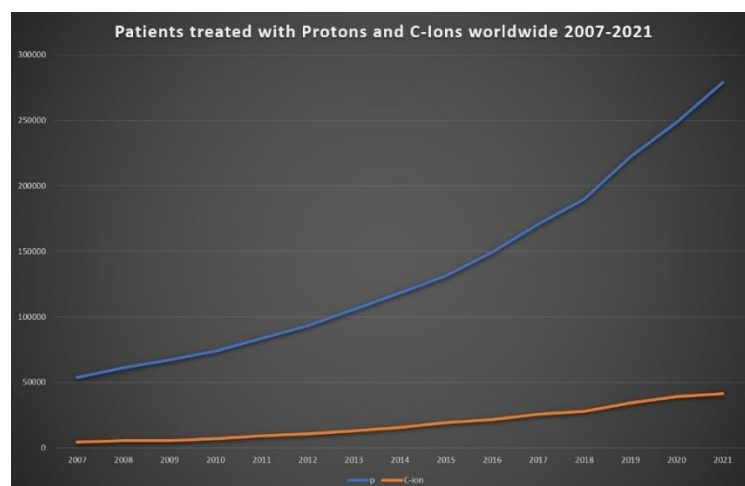


Figure 1 Statistics of patients treated in particle therapy facilities worldwide (Jermann, 2016).

Carbon-ion radiotherapy has superior dose distribution, compared with proton beam therapy, due to its beam characteristic and greater particle mass and energy deposition. The fact that the carbon ions exhibit a so-called Bragg peak in the body allows the delivery of sufficient and precise dose to destroy the cancer cells and to apply less dose to the

surrounding tissues. Moreover, carbon-ions have higher radiobiological effectiveness (RBE), which measures how effective a particular type of radiation is in causing damage to living tissues. Compared with protons, the RBE of carbon-ion beams increases as they penetrate deeper to the tumor region as shown in Figure 2 (Tsuji, 2017).

Because of its superiorities, carbon ion therapy has a wide range of indication for resistant tumors, hypoxic tumors and radiotherapy treatment of recurrent tumors. The indications of carbon ion therapy are for example brain tumor, head and neck tumor, lung cancer, tumor of the digestive system, bone and soft tissue tumors , etc.(Zhang et al., 2021). However, there are relatively few contraindications for ion therapy. Patients with conditions such as systemic diseases, extensive metastases, or serious radiation injury must undergo thorough consultation and assessment before undergoing ion therapy.

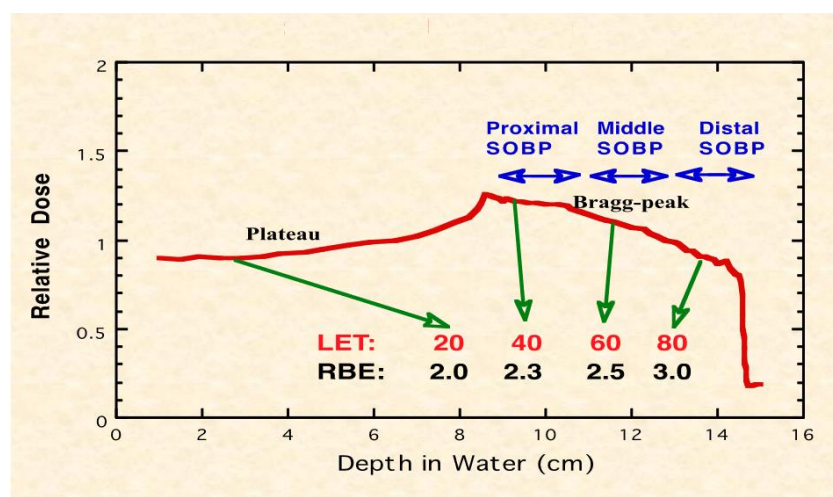


Figure 2 LET and RBE of carbon-ion beams (290MeV, SOBP=60mm) (Tsuji, 2017).

When determining the planning target volume (PTV), it is crucial to meticulously consider the potential uncertainties associated with the range of the radiation beam. The incorporation of an additional margin is a standard practice to address these uncertainties effectively. Depending on the tumor's depth, an extra margin of approximately 0.3 to 0.5 cm is typically applied on the sides of the radiation field. Moreover, along the direction of radiation delivery, a margin of about 0.7 to 1.0 cm is added (Zhang et al., 2021). This

additional margin serves to account for uncertainties related to patient setup variations and potential motion during treatment sessions. These safety margins contribute to the development of a robust treatment plan, enhancing the accuracy and effectiveness of carbon ion therapy.

2.4 Generative Adversarial Networks (GANs)

2.4.1 Related works

Generative Adversarial Networks (GANs) have indeed found diverse applications in various domains, including digital image processing, medical imaging analysis, medical informatics, and bioinformatics (Lan et al., 2020). Various related works in the field of dose computation for radiotherapy used GANs to predict radiation dose to enhance treatment planning efficiency and improve the quality of radiotherapy plans.

(Sarrut et al., 2019) presented a novel approach utilizing a compact GAN to model large phase space files in Monte Carlo simulations. The GAN-generated particles exhibit promising results, with 3D energy distributions closely resembling reference ones, showing less than 1% voxel-by-voxel relative difference.

(Kearney et al., 2020) propose DoseGAN as dose prediction algorithm. While alternative convolutional neural networks (CNNs) use pixel-to-pixel loss to update network parameters, DoseGAN incorporates image-level loss and is better suited to learn from heterogeneous labels. The algorithms were compared using heterogeneity index, conformity index, and various dosimetric parameters. However, there was no depth-dose curves shown to compare the prediction with the MC simulation.

The recent study by (Mentzel et al., 2022) employed a GAN as a proof-of-concept to predict dose in simplified irradiation scenarios involving a water phantom with a rotating bone slab and varying thickness, as well as a pediatric head phantom. It successfully indicates

deviations of less than 3% of the maximum deposited energy from the simulation for approximately 99% of the voxels within the beam field. However, the use of a broad beam instead of a microbeam and the absence of real irradiation scenarios raise questions about the model's performance in more complex and realistic settings. Moreover, none of the above studies explored the application of carbon ion particles.

2.4.2 Architecture & Concept

GANs comprise two primary components: the generator (G) and the discriminator (D) which are trained in a game-like fashion. The two models compete with each other. That is why the name Adversarial was chosen (Goodfellow et al., 2014). The generator takes random noise as input and endeavors to generate data samples resembling real data. The discriminator works as a classifier and aims to distinguish between real data samples from the training set and fake data samples generated by the generator.

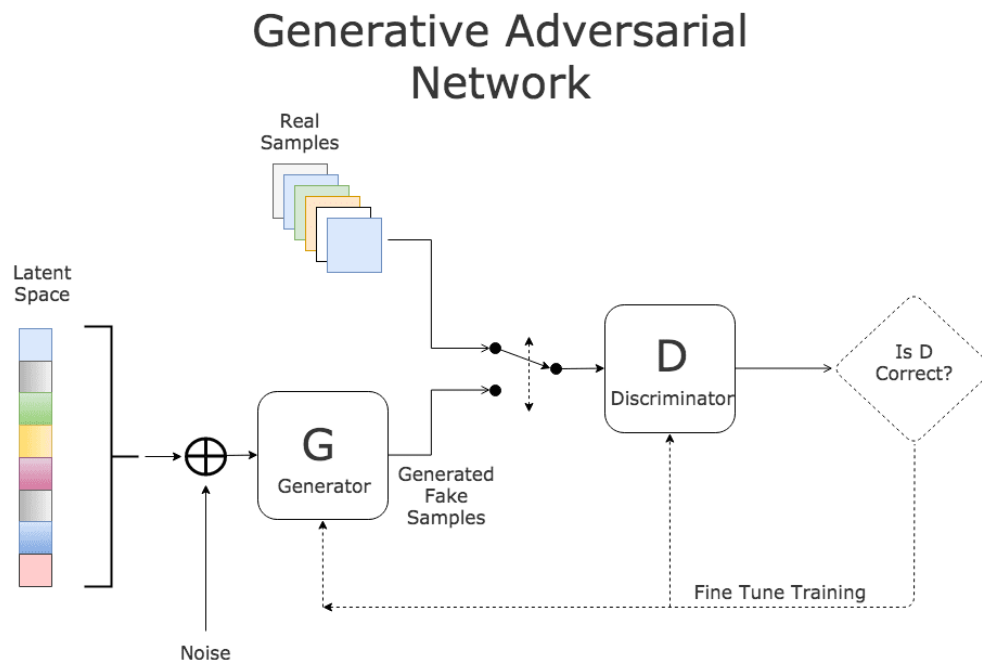


Figure 3 Architecture of a generative adversarial network. (Image source: [KDNuggets](#)).

In the realm of Generative Adversarial Networks (GANs), the training process is driven by a loss function that compels the generator to produce realistic data, fooling the discriminator. This loss function involves two components: the discriminator loss and the generator loss. GANs operate as a minimax game, where the generator seeks to minimize its loss, while the discriminator aims to maximize its loss. The objective is to train the generator to produce realistic data samples capable of deceiving the discriminator.

Throughout the training process, the discriminator and the generator undergo alternate updates in a series of mini-batch iterations. Each iteration involves feeding a batch of real data samples and a batch of noise samples. As the training progresses, these models compete against each other and learn from each other. The generator is getting better in creating realistic images, while the discriminator improves its ability to accurately classify them.

A discriminative model is similar to the one typically used for classification in machine learning. They learn how to distinguish between classes and are often called classifiers. Discriminative models take a set of features X , and from these features determine a category. In other words, they try to model the probability of class Y given a set of features X . Generative models, on the other hand, try to learn how to make a realistic representation of some class. Generative models try to capture the probability distribution of X and generate more pictures representing the dataset that the model was trained on.

Traditionally, in the early days of training Generative Adversarial Networks (GANs), the optimization process heavily leaned towards minimizing the Kullback-Leibler (KL) divergence between two probability distributions. The KL divergence is a measure of how one probability distribution diverges from a second, expected probability distribution. However, training GANs became more successful when using a different divergence measure, namely the Jensen-Shannon (JS) divergence. The Jensen-Shannon divergence is a symmetric and smoothed version of the KL divergence and provides a more balanced measure of the similarity between two probability distributions. (Weng, 2019)

The loss function is based on Binary Cross Entropy function (BCE) function loss. It is optimized so that, on the one hand, the discriminator is trained to maximize this term. On the other hand, the generator is trained to minimize it. The minimax objective is the following: (Weng, 2019)

$$\begin{aligned} \min_G \max_D L(G, D) &= E_{x \sim p_r} [\log D(x)] - E_{z \sim p_z} [\log(1 - D(G(z)))] \\ &= E_{x \sim p_r} [\log D(x)] - E_{x \sim p_g} [\log(1 - D(x))] \end{aligned} \quad \text{Eq. 4}$$

where p_z is the data distribution over the noise input z which is usually uniform or Gaussian distribution, p_g is the generator's distribution over data x , and p_r is the data distribution over real samples x . $D(x)$ is the output of the discriminator when given a real data sample x . And $G(z)$ is the output of the generator when given a random noise vector z .

2.4.3 Limitations & Challenges

There are well known proven limitation of the GAN architecture, which are mode collapse, vanishing gradient, instability, and non-convergence (Arjovsky & Bottou, 2017). Mode collapse occurs when the generator fixates on generating a limited set of data patterns that successfully deceive the discriminator. It becomes fixated on a few dominant modes within the training data, failing to capture the full diversity of the data distribution. Just as the artist refrains from exploring new styles, the generator in GANs avoids traversing the entire data distribution. This results in the generation of repetitive data samples, overlooking the rich variations present in the real data. Overtime during training, the generator can produce less variety of images because it learns that it can get high score for some of the images. This makes training a GAN difficult.

Vanishing gradient occurs when p_r and p_g are disjoint. If the discriminator is perfect, the gradient of the loss function approaches zero. There is no gradient to update the loss during training iterations. This results in extremely slow or even stopped learning. Conversely, when the discriminator performs poorly, the generator lacks precise feedback, leading to a

loss function that fails to accurately represent reality. This dilemma significantly complicates the GAN training process.

2.4.4 Improvement

In response to numerous known limitations and challenges within GANs, various iterations and enhancements have been introduced to improve their functionality and address inherent issues. Some related and notable improvements made are the following:

Conditional Generative Adversarial Networks (CGAN) proposed by (Mirza & Osindero, 2014) in 2014, represent an improvement from the original GAN architecture. CGAN introduces the capability to condition the network by incorporating additional information, such as class labels, during training. This conditioning enables the model to learn specific correlations between input images and their associated labels. Unlike the unsupervised nature of the original GAN, CGAN employs a supervised approach, enhancing the control over the generated results. By providing the generator with both random noise and category labels and feeding the discriminator with the generated or real samples along with their corresponding labels, the CGAN architecture facilitates a more controlled and guided data generation process. This structured approach allows to instruct the model to generate images based on specific input conditions, enhancing the applicability of CGAN in various contexts. In this project, the CGAN is leveraged by concatenating the phantom density matrix and the dose map in water phantom as conditional inputs.

Deep Convolutional Generative Adversarial Networks (DCGAN) introduced by (Radford et al., 2016) in 2015 after the first GAN paper was published. DCGAN enhanced the GAN architecture by incorporating deep convolutional networks (CNNs). This modification revolutionized the field, establishing DCGAN as an important milestone in GAN history. DCGAN's network structure remains prevalent and widely adopted, characterized by the extensive utilization of convolution layers instead of fully connected layers. The discriminator and generator exhibit near symmetry in their architecture, and the network

excludes pooling and up-sampling layers. The integration of the Batch Normalization algorithm in DCGAN mitigated the challenge of vanishing gradients (Radford et al., 2016).

Nevertheless, the main reason behind non-convergence is that the divergence used by the classic GAN does not account for "distance". Therefore, it is difficult to get a meaningful gradient when the generator is not performing well. The gradient should guide the generator to move closer to the real probability distribution, but it does not know that information.

As researchers delved deeper into the complexities of GANs, various techniques have been proposed such as feature matching, minibatch discrimination, historical averaging (Salimans et al., 2016). (Arjovsky & Bottou, 2017) proposed methods to solve the problem of disjoint distributions. Adding noise can help to spread out the two probabilities distribution to overlap. Lastly, it is proposed by (Arjovsky et al., 2017) to use a better and more meaningful metric than the JS divergence in the DCGAN, which fails to give meaningful value in the case that two distribution do not overlap. The Wasserstein distance, which will be covered thoroughly in the next session, offers such an approach to prevent mode collapse and is proposed to substitute JS divergence.

2.5 Wasserstein GAN

The Wasserstein GAN (WGAN) was proposed by (Arjovsky et al., 2017) as an alternative to improve the training of GANs. It extends traditional GANs by employing the Wasserstein distance as the loss function instead of the conventional cross-entropy. This modification provides a more stable and informative training signal, promoting smoother learning and reducing the likelihood of mode collapse. The gradient of the Wasserstein distance facilitates improved convergence, making WGANs effective in handling mode collapse and generating diverse and realistic samples (Arjovsky et al., 2017).

A new metric called Wasserstein distance can tell how far apart the bits of probability distributions are in Euclidean space. Wasserstein distance, also known as the Earth Mover's distance, tells how far we must go to transport points from one distribution to another. Using the Wasserstein loss function is like finding the optimal plan in moving the points in p_r into p_g . The distance formula is:

$$W(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} E_{(x,y) \sim \gamma} [|x - y|] \quad \text{Eq. 5}$$

when $\Pi(P_r, P_g)$ is the set of all possible joint probability distributions between p_r and p_g . In this definition, we are interested in the smallest cost. However, exhaustively exploring all potential joint distributions in $\Pi(P_r, P_g)$ to compute $\inf_{\gamma \in \Pi(P_r, P_g)}$ is impractical. Therefore, (Arjovsky et al., 2017) suggest a transformation of the formula using Kantorovich-Rubinstein duality to derive the WGAN value function :

$$W(P_r, P_g) = \sup_{|f|_L \leq 1} (E_{x \sim P_r}[f(x)] - E_{x \sim P_g}[f(x)]) \quad \text{Eq. 6}$$

Here we want to measure the supremum or the maximum value. There is though one condition that the discriminator or critic must fullfill, namely it must be 1-Lipschitz continuous. Then, we can rewrite the Wasserstein distance as:

$$W(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} E_{(x,y) \sim \gamma} [f(x) - f(y)] \quad \text{Eq. 7}$$

There are mainly two possibilities to enforce this requirement. The first proposed solution by the author of the Wasserstein GAN (Arjovsky et al., 2017) to enforce Lipschitz Continuity is through weight clipping. Although mentioned to be a crude and unoptimized solution, the method is as straightforward as clipping the weight of the critic to lie within a small range $[-c, c]$. The threshold c must be carefully tuned (Arjovsky et al., 2017).

However, (Gulrajani et al., 2017) demonstrated some problematic behavior observed such as vanishing or exploding gradients when implementing weight clipping. These can lead to optimization difficulties and WGAN critics often failing to converge. The author proposed another solution called gradient penalty (WGAN-GP) to regularize and stabilize the weight updates. The intuitive of gradient penalty is that it ensures the norm of the gradient or slope to be at most 1 for every point. The function will be bounded to grow linearly because this will help increase the stability.

The goal of a WGAN is to minimize the Wasserstein distance, which is a measure of the distance between the distribution of the real data and the distribution of the generated data. We can express **Eq. 7** in terms of the generator G and the critic D of the GAN. Let $G(z)$ be the generated sample, where z is drawn from a noise distribution. The WGAN value function is then defined as:

$$V(G, D) = E_{x \sim p_r}[D(x)] - E_{z \sim p_z}[D(G(z))] \quad \text{Eq. 8}$$

Where p_z is the noise distribution.

The critic is trained to output values that approximate the Wasserstein distance. The Wasserstein distance is not explicitly computed during training, but rather the critic loss is minimized. The critic is trained to maximize this function. It tries to increase the score for real samples and decrease the score for generated samples. This leads to a WGAN training procedure where the critic is trained to be a 1-Lipschitz function.

In practical application, the critic loss L_c is often formulated as the difference between the mean critic score for real samples and the mean critic score for generated samples. It is proportional to the negative Wasserstein distance. Minimizing the critic loss indirectly encourages the generator to produce samples that are closer to the real data distribution. With the gradient penalty term added, we train the critic to minimize the following function:

$$L_c = E[D(x)] - E_{\tilde{x} \sim p_g}[D(\tilde{x})] + \lambda E_{\hat{x} \sim p_{\hat{x}}}[(|\nabla_{\hat{x}} D(\hat{x})|_2 - 1)^2] \quad \text{Eq. 9}$$

The gradient penalty penalizes the critic if the gradients of its output with respect to the input $\hat{x}$ deviate from a norm of 1. The term $\hat{x}$ is sampled from the distribution $p_{\hat{x}}$ which is usually a linear interpolation between real and generated samples. λ is a hyperparameter that controls the importance of the gradient penalty term relative to the other terms in the loss. $|\nabla_{\hat{x}} D(\hat{x})|_2$ is the L2 norm of the gradient of the critic's scores with respect to $\hat{x}$.

The generator aims to minimize the value function in **Eq. 8**. Its goal is to produce data that receives a high score from the critic or is considered real. It aims to minimize the following loss function:

$$L_g = -E_{\tilde{x} \sim p_g}[D(\tilde{x})] \quad \text{Eq. 10}$$

The main benefits of a WGAN are that we have a meaningful loss metric that correlates with the generator's convergence and improved stability of the optimization process. While the discriminator of a classic GAN saturates and results in vanishing gradients, a WGAN critic provides very clean gradients across a wide range of conditions or input scenarios. It does not limit the output to be between zero and one. Thus it is not a direct critic of telling the fake samples apart from the real ones anymore. Instead, it is trained to learn a function to help compute the Wasserstein distance.

Keep in mind that while the critic loss is related to the Wasserstein distance, it does not directly provide a numerical value for the Wasserstein distance. During training, we can monitor the critic loss as a proxy for the Wasserstein distance. For a more accurate evaluation, dedicated tools or libraries that specifically compute the Wasserstein distance can be used.

Algorithm 1 WGAN with gradient penalty. We use default values of $\lambda = 10$, $n_{\text{critic}} = 5$, $\alpha = 0.0001$, $\beta_1 = 0$, $\beta_2 = 0.9$.

Require: The gradient penalty coefficient λ , the number of critic iterations per generator iteration n_{critic} , the batch size m , Adam hyperparameters α, β_1, β_2 .

Require: initial critic parameters w_0 , initial generator parameters θ_0 .

```

1: while  $\theta$  has not converged do
2:   for  $t = 1, \dots, n_{\text{critic}}$  do
3:     for  $i = 1, \dots, m$  do
4:       Sample real data  $\mathbf{x} \sim \mathbb{P}_r$ , latent variable  $\mathbf{z} \sim p(\mathbf{z})$ , a random number  $\epsilon \sim U[0, 1]$ .
5:        $\tilde{\mathbf{x}} \leftarrow G_{\theta}(\mathbf{z})$ 
6:        $\hat{\mathbf{x}} \leftarrow \epsilon \mathbf{x} + (1 - \epsilon) \tilde{\mathbf{x}}$ 
7:        $L^{(i)} \leftarrow D_w(\tilde{\mathbf{x}}) - D_w(\mathbf{x}) + \lambda(\|\nabla_{\hat{\mathbf{x}}} D_w(\hat{\mathbf{x}})\|_2 - 1)^2$ 
8:     end for
9:      $w \leftarrow \text{Adam}(\nabla_w \frac{1}{m} \sum_{i=1}^m L^{(i)}, w, \alpha, \beta_1, \beta_2)$ 
10:   end for
11:   Sample a batch of latent variables  $\{\mathbf{z}^{(i)}\}_{i=1}^m \sim p(\mathbf{z})$ .
12:    $\theta \leftarrow \text{Adam}(\nabla_{\theta} \frac{1}{m} \sum_{i=1}^m -D_w(G_{\theta}(\mathbf{z})), \theta, \alpha, \beta_1, \beta_2)$ 
13: end while

```

Figure 4 Pseudocode of WGAN with gradient penalty (Gulrajani et al., 2017)

3 Material and Methods

3.1 Conception

The development of a machine learning (ML) model using PyTorch followed a systematic and iterative process aimed at achieving optimal performance. The conception phase involved a series of key steps:

Data Preparation:

The initial step was to gather, clean, and preprocess the data. This involved tasks such as data conversion, handling missing values, resampling, scaling, and splitting the dataset into training and testing sets. The quality of the model heavily depends on the quality of the data used for training.

Model Construction:

The next crucial aspect was building the model architecture. This step included defining the neural network's structure, selecting appropriate activation functions, and determining the number and type of layers.

Model Training:

Once the model architecture was in place, the model was trained using the prepared dataset. During training, the model learned to map input data to the corresponding output by adjusting its internal parameters. PyTorch's flexibility allows for seamless implementation of this training process.

Improvement through Experimentation and Hyperparameter Tuning:

Continuous improvement of the model's performance was achieved through experimentation. Hyperparameter tuning involved adjusting parameters such as learning rate, batch size, and the parameter λ (see *Eq. 9*), which controls the importance of the gradient penalty term. This iterative process aimed to find the optimal configuration for the model. Validation was performed using a dedicated validation dataset, ensuring that the model's performance was rigorously assessed on unseen data.

Model Evaluation:

After training, the model's performance was assessed using a separate test dataset not seen during training. Evaluation metric such as loss or Mean Square Error was used to assess how well the model generalizes to new, unseen data.

Save and Reload the Model:

Once satisfied with the model's performance, the trained model's weights and architecture were saved. This step ensured that the model can be easily reloaded for future use, eliminating the need to retrain from scratch. It facilitated the seamless integration of the model into production environments and enabled its deployment for real-world applications.

In essence, the conception phase sets the foundation for a well-structured and effective ML model, guiding the subsequent steps in the development process.

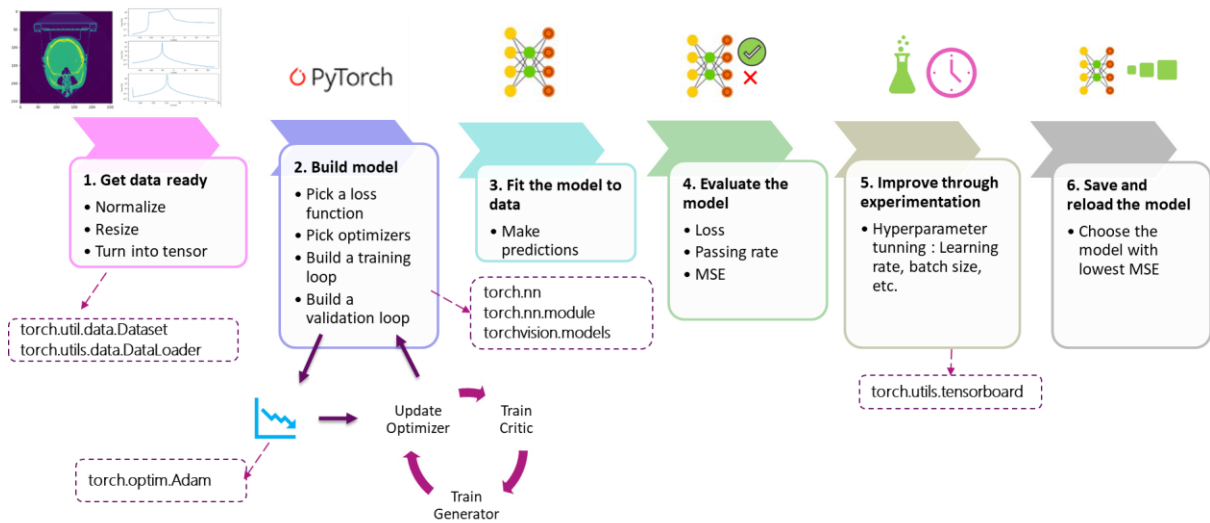


Figure 5 Steps in the ML model development process.

3.2 Hardware & Software

The small Slurm-based high-performance computing (HPC) environment consisted of following hardware.

- 1 GPU node (28-core Skylake, 192 GB RAM, 2x V100 GPUs)
- 1 GPU node (28-core Skylake, 512 GB RAM, 1x P4000 GPU)

These hardwares had been used in model training and verification. To get more detailed information, additional Slurm commands (see in Figure 6) can be used to provide detailed information about the GPUs on the node, including GPU model, memory usage. The output indicates that GPU resources were allocated for this job, and it were allocated on a node where GPU resources are available.

```
e00072
Tue Jan 16 13:43:05 2024
+-----+
| NVIDIA-SMI 545.23.08                  Driver Version: 545.23.08   CUDA Version: 12.3   |
+-----+
| GPU  Name                Persistence-M   Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|=====+-----+=====+=====+=====+=====+=====+=====+=====+
|  0   Quadro P4000             Off        00000000:2F:00.0 Off |      0MiB /  8192MiB |      1%      Default |
| 36%   30C    P0              26W / 105W |      0MiB /  8192MiB |      1%      Default |
|=====+-----+=====+=====+=====+=====+=====+=====+=====+
+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type   Process name                  GPU Memory |
|      ID     ID                                   |            Usage            |
|=====+-----+=====+=====+=====+=====+=====+=====+=====+
|  No running processes found              |
+-----+
0

HOSTNAME = e00072
Running on 14 processor(s) on 1 node(s).
Running GPUs:
Tue Jan 16 13:43:05 CET 2024

1.12.1
Device : cuda

tappay01@node49:~/program>scontrol show node e00072
NodeName=e00072 Arch=x86_64 CoresPerSocket=14
CPUAlloc=14 CPUTot=28 CPULoad=1.08
AvailableFeatures=(null)
ActiveFeatures=(null)
Gres=gpu:P4000:1
NodeAddr=e00072 NodeHostName=e00072 Version=20.11.8
OS=Linux 4.18.0-305.3.1.el8_4.x86_64 #1 SMP Thu Jun 17 07:52:48 UTC 2021
RealMemory=510000 AllocMem=100000 FreeMem=483311 Sockets=2 Boards=1
State=MIXED ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_Label=N/A
Partitions=equipment_typeG,all
BootTime=2024-01-12T16:58:32 SlurmdStartTime=2024-01-12T17:41:36
CfgTRES=cpu=28,mem=510000M,billing=28
AllocTRES=cpu=14,mem=100000M
CapWatts=n/a
CurrentWatts=0 AveWatts=0
ExtSensorsJoules=n/s ExtSensorsWatts=0 ExtSensorsTemp=n/s
Comment=(null)
```

Figure 6 nvidia-smi tools on the node showing GPU information (top). Detailed information about the compute node, including its hardware specifications (bottom).

The machine learning model was implemented using the following software and framework:

- Python 3.7.12: The primary programming language for model development and training.
- PyTorch 1.12.1: An alternative machine learning framework used for model development and training, offering dynamic computational graphs and a strong focus on flexibility and research.
- NumPy 1.21.6: Used for numerical computations and array operations.
- Pandas 1.3.5: Employed for data manipulation and analysis.

- scikit-learn 0.21.2: Utilized for additional machine learning utilities such as train-test splitting and performance evaluation.

All Python packages were installed within a virtual environment to manage dependencies.

3.3 MC Simulation

Monte-Carlo radiation transport simulations were performed using the simulation software Geant4 (Agostinelli et al., 2003) to calculate dose distributions. Geant4 version 11.0.2 was employed. Pairs of simulations were performed of a carbon ion beam irradiation in a head and a water phantom, respectively. The carbon ion beam was configured as a circle with a 1 mm diameter. Simulations had been generated as part of the dataset “MC Simulation of a therapeutic carbon ion beam in head” by (Schwarze, 2023). All information on the simulation geometry can be found in appendix [A. Dataset: MC Simulation of a therapeutic carbon ion beam in head](#).

The available simulated datasets from the MC simulation and digital phantoms were therefore the following:

- 3D representation of a material density matrix of the head phantom
- 3D matrix of the energy deposition of the carbon ion beam in a head and water phantom.

When comparing the dose maps between the brain simulation and the water simulation, it must be considered that in the human head, carbon ions may exhibit a shorter range and modified Bragg peak depth compared to pure water.

3.3.1 Head phantom

Dataset *Glioma Image Segmentation for Radiotherapy: RT targets, barriers to cancer spread, and organs at risk (GLIS-RT)* (Shusharina & Bortfeld, 2021) was the basis for the dataset simulation for this project. It contains CT images of the head and neck region of various

patients with a brain tumor. Due to the fact that treatment planning is based on CT images, which have information about the density in Hounsfield units, the training dataset also used CT images as geometry. Six patients were randomly selected from the GLIS-RT data set. The voxel size was defined by the voxel size of the CT image, thus they can be different for each image.

The output of the MC simulation was organized as list files containing lines with [voxel index, dose value] ordered ascending in voxel index which encodes the x, y, z coordinates. There were 6 patient datasets available for the project and each has a different number of voxels and different source locations to provide geometric variety. To match a predefined voxel grid system, the data required transformation. This process involved converting the energy deposition into a three-dimensional dose distribution. The result of the conversion can be visualized as shown in Figure 7, illustrating the dose distribution in three different mutually orthogonal lines and depicting the Bragg peak in the beam direction. The code presented in appendix C. [Dose Map](#) was designed for this purpose, to convert linear indices into the corresponding three-dimensional coordinates within a voxel grid system.

Figure 8 shows 2D slices of the material density maps from the CT images for the six datasets and overlaid the positions of the source (left) and the beam paths in the heads. The beam is in the x-direction from left to right (middle and right images). Source positions are shown as red circles in the pictures. All configuration detail about the CT images used in the datasets are listed in appendix B.

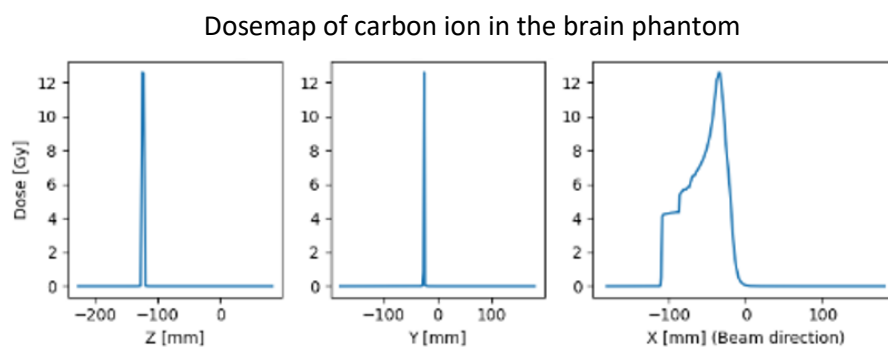
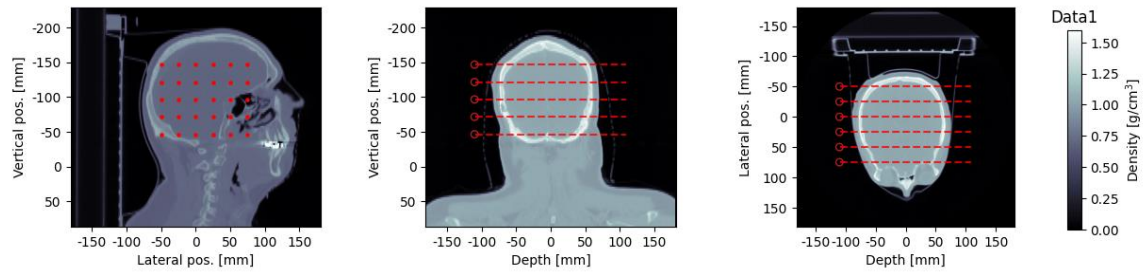
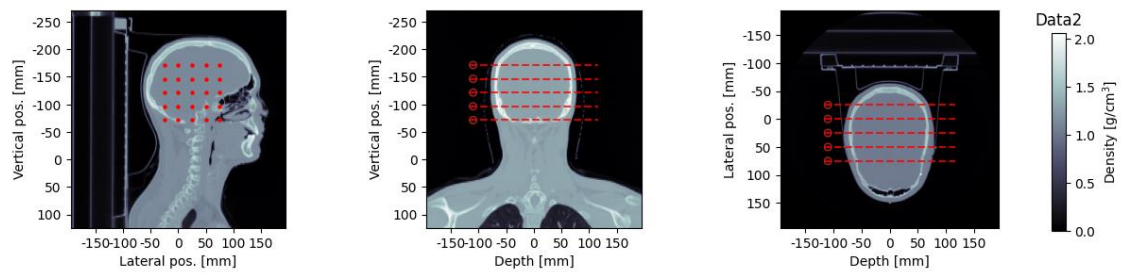


Figure 7 Profiles of the calculated dose distribution of carbon ion in the head along three directions.

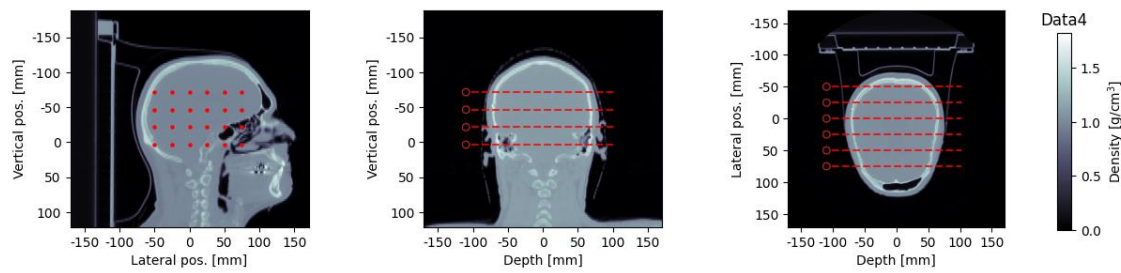
Dataset1



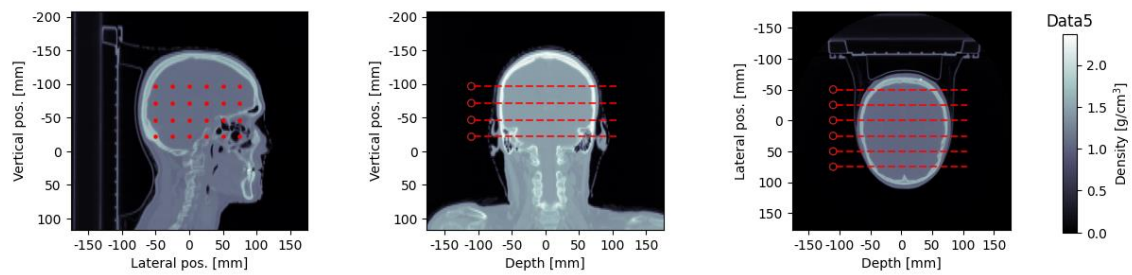
Dataset2



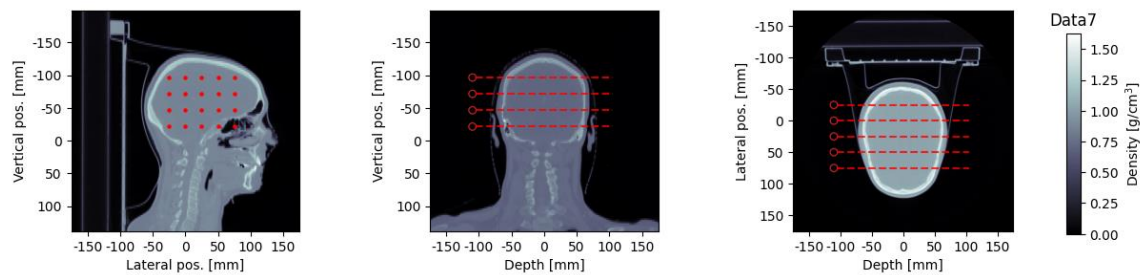
Dataset4



Dataset5



Dataset7



Dataset8

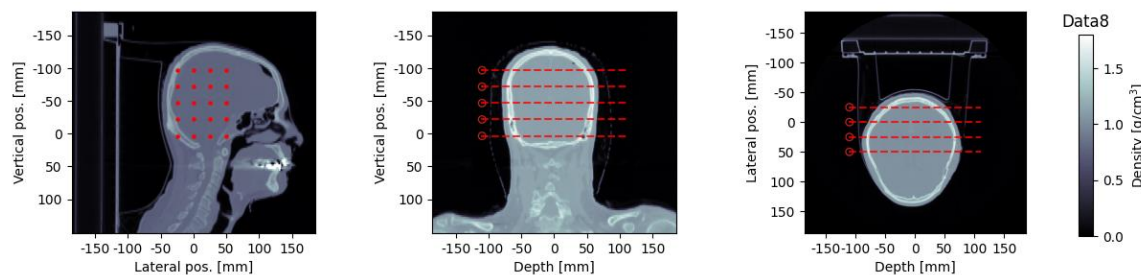


Figure 8 Location of the beam in the lateral and vertical position combinations used in the MC simulations (left). Beam in x-direction from left to right of the head shown as red dash lines (middle, right).

3.3.2 Water phantom

Examining the 3D energy depositions of a beam within a pure water phantom provides crucial information about properties of the beam, including its energy and depth-dose profile. This simulated data offered neural network information about the beam's interaction with a default material (water), presented in the same format as the brain simulation. Utilizing this knowledge as input could enhance the model's capability to learn and generate more precise predictions.

Energy deposition in a water phantom from heavy ion interactions was simulated and saved as raw csv data. The generated simulation output data was processed to convert the raw simulation data into a three-dimensional dose map. The dose calculation was done for each carbon ion energy in the dataset. The water phantom had a size of 400 mm x 400 mm x 400 mm. The number of voxels in each dimension was calculated based on the phantom size and voxel dimensions. The energy deposition values (in unit MeV) were converted to absorbed

dose (in unit Gy), where the gray is the unit of absorbed dose in the International System of Units (SI). The conversion involved scaling the energy values by the elementary charge and dividing by the mass of water in each voxel.

$$dose [Gy] = \frac{energy [MeV] \times (1 \times 10^6) \times e [C]}{m [kg]} \quad \text{Eq. 11}$$

where $e = 1.602176634 \times 10^{-19}$ denotes the elementary charge in coulombs. m is the mass of water in each voxel (in kg), taking into account the density of water and the dimensions of the voxels. The dose in Gray was saved as a NumPy array. The calculation provided an automated pipeline for processing simulation output data, converting it into meaningful dose maps for further analysis. A sample of the calculated dose array was plotted in three slices in Figure 9, illustrating the Bragg peak position along the beam depth (X-axis) and emphasizing the dose distribution along lateral (Y-axis) and vertical positions (Z-axis).

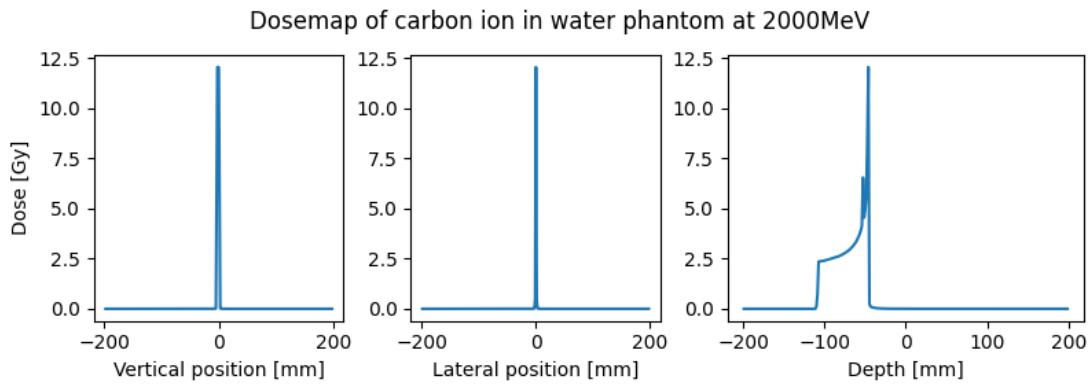


Figure 9 Profiles of the calculated dose distribution of carbon ion in water at 2000 MeV along three directions.

The depth-dose profiles of carbon ion beams at all beam energies are plotted in Figure 10, with doses normalized relative to their respective maximum values. This normalization facilitates a straightforward comparison of the Bragg peak locations.

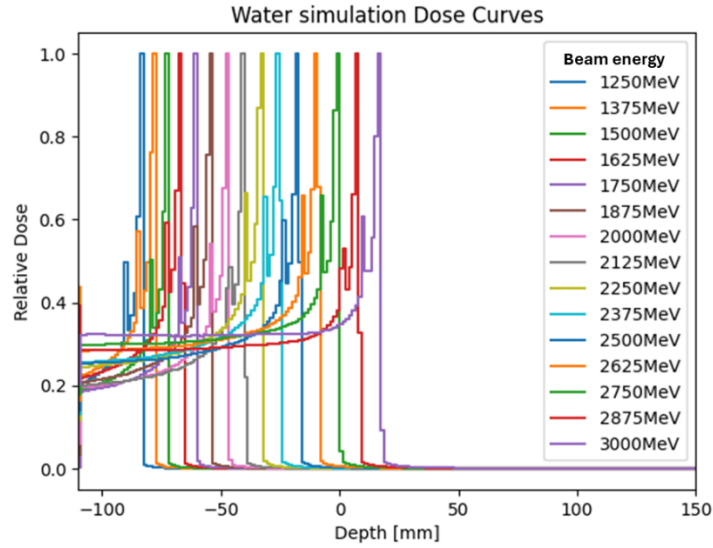


Figure 10 The simulated depth-dose profiles of carbon ion beams in water phantom at different beam energies.

3.4 Data Preprocessing

In the initial phase of data preprocessing, the raw data was transformed into a 3D Numpy array, to facilitate efficient manipulation and analysis. This section explains steps in preparing the heavy ion simulation data for further analysis. Resampling ensures consistency across the different datasets, and regions of interest (ROI) extraction allows the focus on specific ROIs for efficient model training.

3.4.1 Resampling

Before utilizing datasets for GAN training, it was crucial to ensure their consistency in terms of voxel size and range. This ensured that the trained model generalizes well and did not learn artifacts that could arise, e.g., due to a difference in voxel resolutions.

Each dataset comes with specifications such as voxel dimensions, origin, positions, and voxel center. To establish uniformity, a common voxel size needs to be chosen for all datasets. A practical approach is to select the voxel size from the dataset with the highest resolution (i.e., the smallest voxel size) to preserve maximum information. The SimpleITK library

(<https://simpleitk.org/>) was employed for resampling the datasets. The data, initially stored as a numpy array, was converted into a SimpleITK image, which was then resampled to align with the physical space of a target dataset. To mitigate the potential loss of information and ensure uniformity across datasets, the smallest voxel sizes of 2.5, 1.33, 1.33 mm was chosen in the z, y, x coordinates, respectively, as the target for resampling (see appendix B).

3.4.2 Cropping datasets to focus on a specific region of interest (ROI)

The subsequent step involved cropping the data to precisely determine the beam center. It was mainly driven by the goal to optimize computational efficiency and reduce computational resource demands. In defining the simulation space, eight lateral voxels from the source position were considered, spanning a total of 256 voxels along the trajectory through the head. The preprocessing algorithm establishes the size of the region of interest and identifies source positions along the y and z axes. For every unique combination of these positions, the algorithm extracted an ROI from the resampled image with the beam position in the center.

This approach enabled a refined focus on crucial details related to the beam's interaction with the simulated environment, thereby contributing to a more efficient utilization of computational resources.

3.4.3 Shuffle and split

At this stage it is possible to combine and shuffle all the various datasets. Shuffling the combined data is essential during training to prevent models from being biased by extended sequences from a specific dataset. It's essential to maintain a clear separation between training data and validation or test data. This is to ensure that the model's performance is evaluated on data it has not seen during training, providing a more accurate estimation of its ability to generalize to new, unseen data. To facilitate training and evaluation, the dataset was split into three subsets with a ratio of 70:15:15:

Training set: Used for training the model.

Validation set: Utilized for tuning hyperparameters and assessing the model's performance during training.

Test set: Held out and employed only after the model has been trained and tuned, serving to evaluate its performance on new, unseen data.

3.4.4 Create custom dataset and Dataloader

Subsequently, a custom dataset class was created which is inheriting from the `torch.utils.data.Dataset` module. This class effectively organizes and manages both head and water simulation data, including the density information. This custom dataset class encapsulated the necessary preprocessing steps for the carbon ion simulation data, providing flexibility in data augmentation and normalization strategies. It established an organized interface for handling input data as follows:

Augmentation:

Considering the relatively small size of the datasets, data augmentation techniques were applied to artificially increase the training dataset's size. When augmentation was enabled, additional samples were generated by rotating the original simulation data in the head phantoms and water, and the density arrays by angles of 90 and 180 degrees. This approach enhanced the number of training samples and utilizes the `rotate` function from the `scipy.ndimage` module.

Initialization:

The constructor `__init__` initializes the dataset with paths to simulation data, density maps, and water simulation results. Additional parameters include normalization strategy, augmentation flag, and default values for voxel dimensions and origin.

Data Loading:

Data was loaded from numpy files, and water simulation and density samples were extracted accordingly.

Thresholding:

Thresholds were applied to the loaded data and density samples. Values below a dose threshold are set to 0, and density values exceeding a specified threshold were capped as they can be treated as outliers.

Normalization:

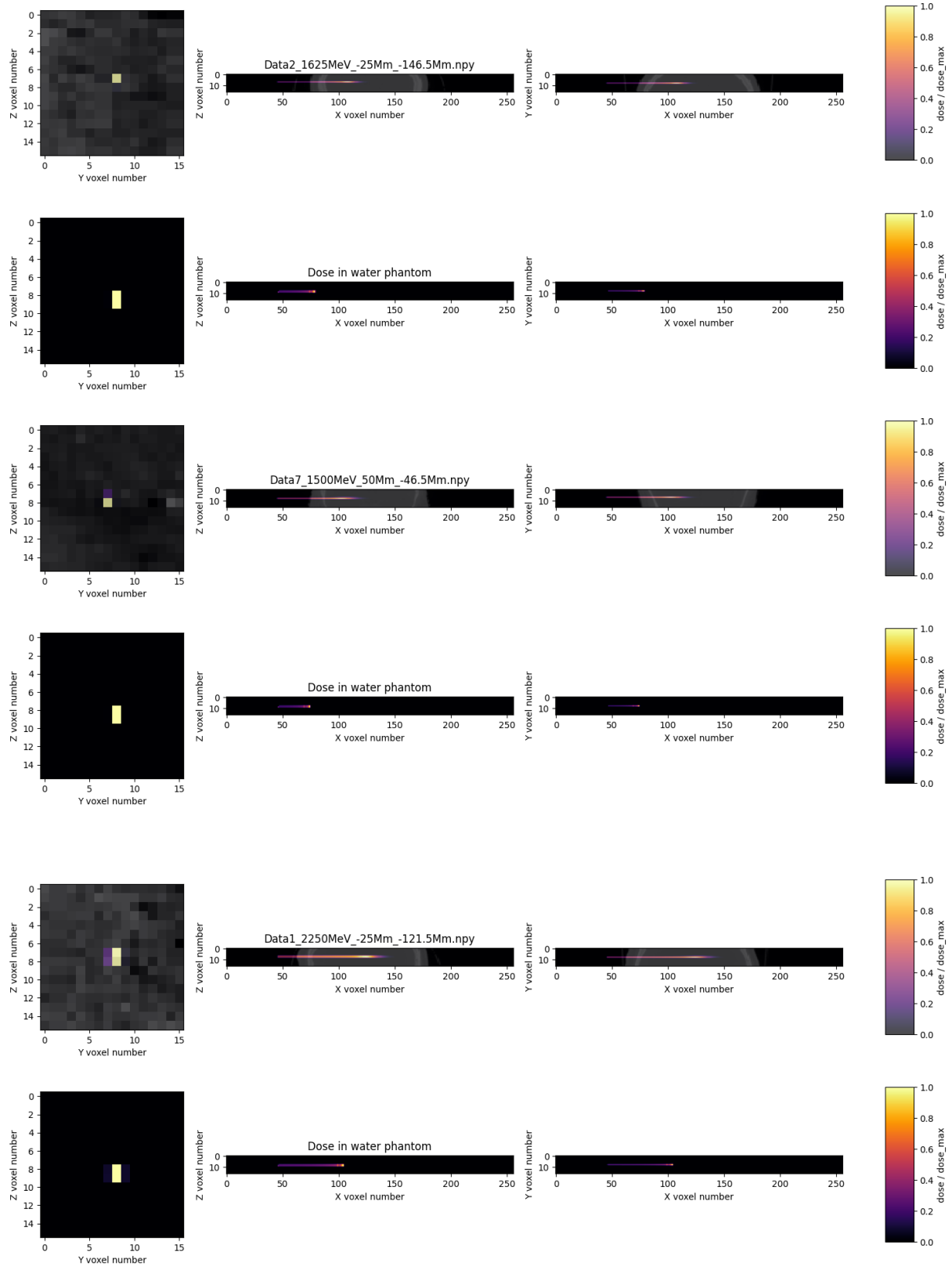
Data and density samples were normalized using the maximum value within each sample. Min-max normalization was applied across all datasets to ensure a consistent range [0, 1], preventing the GAN from learning artifacts due to intensity differences in the datasets.

Item Retrieval:

The `__getitem__` method retrieves a specific item or sample from the dataset. As a final step, the data was converted into tensors to ensure compatibility with the Pytorch machine learning framework. The method returned torch tensors for data, density, water simulation, and the corresponding data file name.

Finally, to optimize GPU utilization, a DataLoader was employed to load the data in batches of 32 samples. Visualizing the cropped array, sized at 16x16x256, provides a clear depiction of a specific area of interest within the simulation. In Figure 11, samples from different datasets in the same physical space were plotted and compared with their respective doses in the water phantom. The visual representation aids in understanding variations in dose distribution across datasets. Plotting the normalized dose reveals that the highest dose is concentrated at the center of the ROI along the beam center. This comprehensive data preprocessing workflow ensures that the data is prepared for training and evaluating dose prediction models.

Samples from preprocessed datasets compared with their respective doses in water.



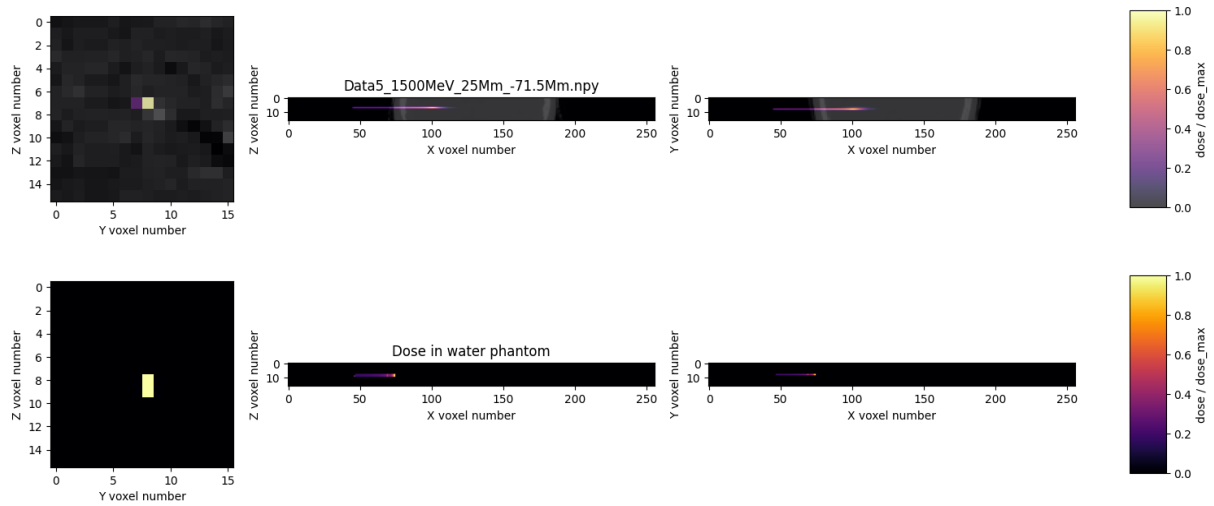


Figure 11 Samples from different datasets compared with their respective doses in the water.

4 Implementation

4.1 The generator model

The generator has 3D-UNet architecture and a conditional element to generate 3D images. The architecture, as drawn in Figure 12, is a type of convolutional neural network (CNN) that is designed for 3D image segmentation tasks. It is composed of:

- An encoder network that extracts features from the input 3D image.
- A decoder network that generates a segmentation mask.
- Skip connections that allow the network to preserve spatial information.

The encoder comprises five convolutional blocks, each followed by max pooling. The number of channels for these blocks are 2, 16, 32, 64, and 128. Within each block, two sequential convolutional layers with batch normalization, swish activation function, and a 15% dropout rate are applied.

The bottleneck feature extracted by the encoder is concatenated with 100 values of random noise sampled from a normal (Gaussian) distribution. This combination serves as the input for the decoder.

The decoder channels mirror the encoder with sizes 128, 64, 32, and 16. Each decoder block involves convolutional transpose operations and concatenation with feature maps from the corresponding encoder layers (skip connections). The final layer of the decoder employs the same convolutional block.

The generator network takes both a random noise vector and conditional information as input. The conditional input provides additional information to the generator. It outputs a 3D matrix of dose distribution of the same size as the input. A summary of the architecture and parameters of the generator model used for the experiments is given in Figure 13. The input sizes are specified as (32,2,16,16,256) as one 5-dimensional input tensor. The summary includes information on the layers, output shapes, and the total number of parameters.

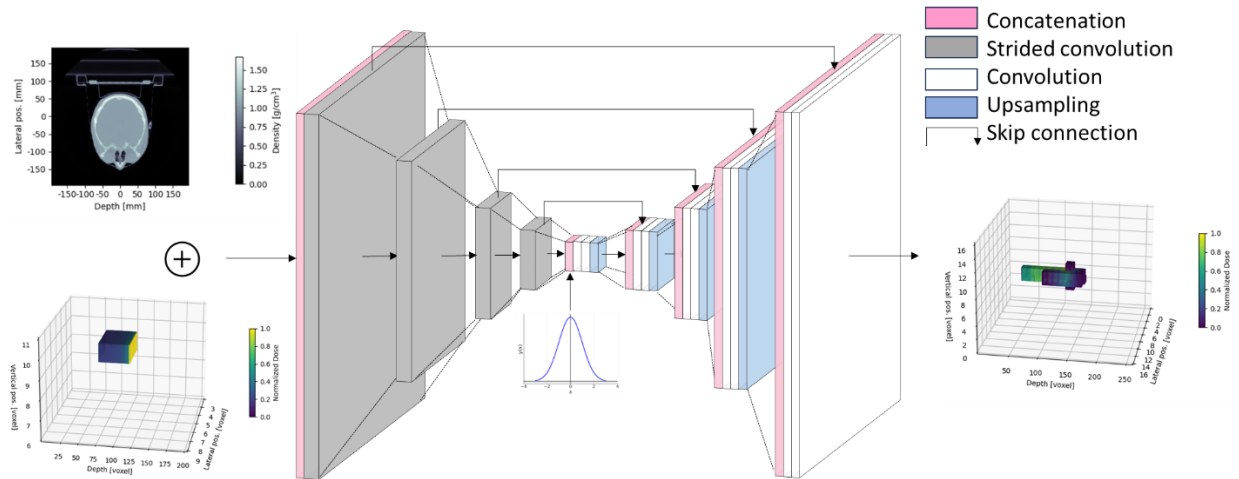


Figure 12 The generator model architecture adapted from (Mentzel et al., 2022).

Layer (type:depth-idx)	Output Shape	Param #
Generator	[32, 1, 16, 16, 256]	--
└Encoder: 1-1	[32, 228, 1, 1, 16]	--
└ModuleList: 2-7	--	(recursive)
└Block: 3-1	[32, 16, 16, 16, 256]	7,872
└MaxPool3d: 2-2	[32, 16, 8, 8, 128]	--
└ModuleList: 2-7	--	(recursive)
└Block: 3-2	[32, 32, 8, 8, 128]	41,664
└MaxPool3d: 2-4	[32, 32, 4, 4, 64]	--
└ModuleList: 2-7	--	(recursive)
└Block: 3-3	[32, 64, 4, 4, 64]	166,272
└MaxPool3d: 2-6	[32, 64, 2, 2, 32]	--
└ModuleList: 2-7	--	(recursive)
└Block: 3-4	[32, 128, 2, 2, 32]	664,320
└MaxPool3d: 2-8	[32, 128, 1, 1, 16]	--
└Decoder: 1-2	[32, 16, 8, 8, 128]	--
└ModuleList: 2-13	--	(recursive)
└ConvTranspose3d: 3-5	[32, 64, 2, 2, 32]	116,800
└ModuleList: 2-14	--	(recursive)
└Block: 3-6	[32, 64, 2, 2, 32]	332,160
└ModuleList: 2-13	--	(recursive)
└ConvTranspose3d: 3-7	[32, 32, 4, 4, 64]	16,416
└ModuleList: 2-14	--	(recursive)
└Block: 3-8	[32, 32, 4, 4, 64]	83,136
└ModuleList: 2-13	--	(recursive)
└ConvTranspose3d: 3-9	[32, 16, 8, 8, 128]	4,112
└ModuleList: 2-14	--	(recursive)
└Block: 3-10	[32, 16, 8, 8, 128]	20,832
└Conv3d: 1-3	[32, 1, 8, 8, 128]	17
Total params: 1,453,601		
Trainable params: 1,453,601		
Non-trainable params: 0		
Total mult-adds (G): 47.04		
Input size (MB): 16.78		
Forward/backward pass size (MB): 1648.36		
Params size (MB): 5.81		
Estimated Total Size (MB): 1670.95		

Figure 13 Generator model summary

4.2 The critic model

The architecture of the critic is a simple classifier using a 3D convolutional neural network. It has 4 convolutional layers with increasing channel sizes, swish activation functions, batch normalization, and a final linear layer to output a scalar value. The critic network receives both the generated images and the conditional input. The input sizes are specified as (32,1,16,16,256) corresponding to simulated or generated data tensors and (32,2,16,16,256) corresponding to two conditional inputs.

The three input matrices are concatenated and passed through consecutive 3D convolutions. The last single linear unit gives a continuous and quantitative rating, which distinguishes simulated from generated samples. The output represents the "realness" or "fakeness" of the input data. Figure 15 displays a summary of the architecture and parameters of the critic model.

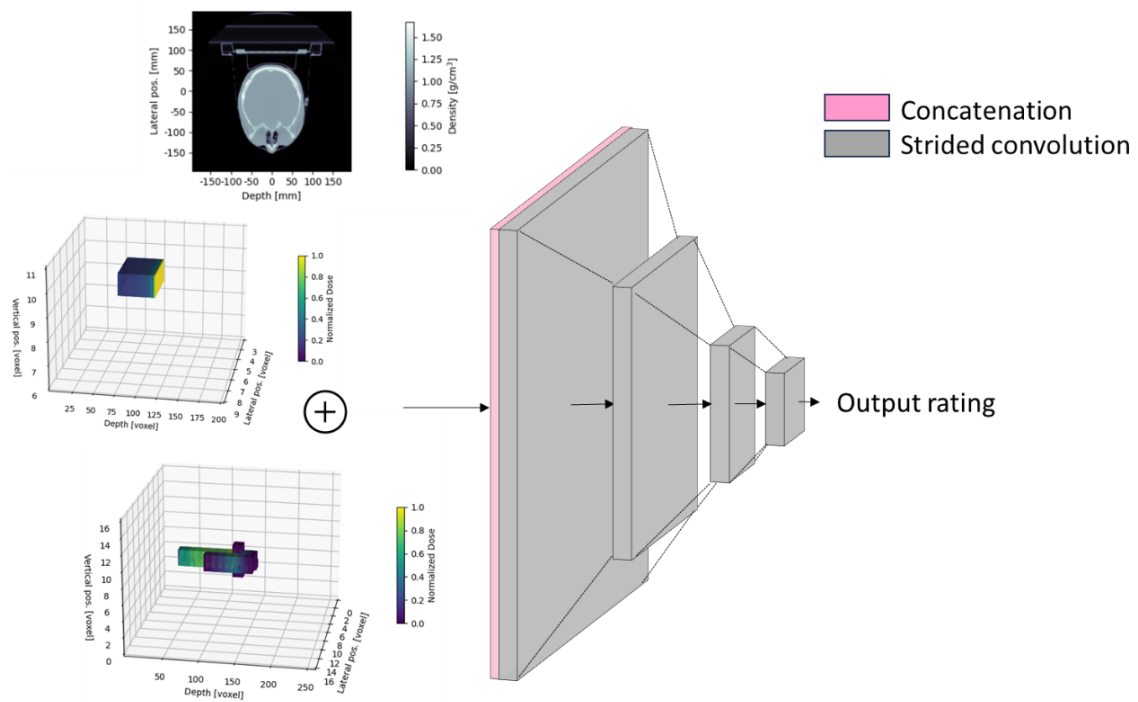


Figure 14 Critic model architecture adapted from (Mentzel et al., 2022).

Layer (type:depth-idx)	Output Shape	Param #
Critic3d	[32, 1]	--
└Sequential: 1-1	[32, 1]	--
└Conv3d: 2-1	[32, 64, 8, 8, 128]	5,184
└SiLU: 2-2	[32, 64, 8, 8, 128]	--
└Conv3d: 2-3	[32, 128, 4, 4, 64]	221,184
└BatchNorm3d: 2-4	[32, 128, 4, 4, 64]	256
└SiLU: 2-5	[32, 128, 4, 4, 64]	--
└Conv3d: 2-6	[32, 256, 2, 2, 32]	884,736
└BatchNorm3d: 2-7	[32, 256, 2, 2, 32]	512
└SiLU: 2-8	[32, 256, 2, 2, 32]	--
└Conv3d: 2-9	[32, 1, 2, 2, 25]	2,048
└Flatten: 2-10	[32, 100]	--
└Linear: 2-11	[32, 1]	101
Total params: 1,114,021		
Trainable params: 1,114,021		
Non-trainable params: 0		
Total mult-adds (G): 12.24		
Input size (MB): 25.17		
Forward/backward pass size (MB): 218.13		
Params size (MB): 4.46		
Estimated Total Size (MB): 247.75		

Figure 15 Critic model summary.

4.3 Model Training

The training procedure encompassed the following key steps:

1. Data Splitting: The dataset was randomly split into training (70%), validation (15%), and test (15%) sets to facilitate a robust model evaluation.
2. Loss Function and Metric: The model was compiled using the Wasserstein loss function and the Mean Squared Error (MSE) was employed as the evaluation metric.
3. GPU Acceleration: Training was conducted in a GPU-accelerated environment to expedite convergence and leverage parallel processing capabilities.

The training of the model involved a meticulous exploration of hyperparameters to achieve optimal performance. The following hyperparameters were considered to investigate their influence on the model's convergence:

- Generator Learning Rate: A grid search was conducted over the range between 0.00001 and 0.009 at the values in the following list: [0.00001, 0.00005, 0.0001,

0.0003, 0.0005, 0.0008, 0.001, 0.002, 0.003, 0.004, 0.005, 0.006, 0.007, 0.008, 0.009].

- Critic Learning Rate: Another grid search was performed, exploring values in the range between 0.000008 and 0.001 at the values in the following list: [0.000008, 0.000009, 0.00001, 0.00002, 0.00003, 0.00005, 0.0001, 0.0005, 0.001].
- Lambda (Gradient Penalty of Wasserstein Loss): A range of values was tested, including the values 1, 5, 10, and 15.
- Critic Iteration: Different numbers of critic iterations, namely 1, 3, 5, and 7, were experimented with.
- Optimizer: Experiments were conducted with both the Adam and RMSprop optimizers.

The "train-validation-test split" method was employed for hyperparameter tuning and model evaluation. The validation set serves as the basis for comparing various hyperparameter configurations and choosing the most effective one. However, after identifying the optimal hyperparameters, the model underwent retraining using both the training and validation sets. This practice aims to ensure that the model learns from a comprehensive dataset, enhancing its ability to generalize effectively.

4.4 Performance measurement

Although the loss value from the ML network can be indicative of the level of performance of the network, it is not sufficient to be used in training a GAN. In the first applications of this approach in microbeam radiation therapy (MRT), the delta index, which is inspired by global gamma index, was used as performance index (Mentzel et al., 2022).

$$\delta = \frac{D_{\text{gen}} - D_{\text{sim}}}{D_{\text{max_sim}}} \quad \text{Eq. 12}$$

where D_{gen} is the dose predicted by the generator network of the GAN, D_{sim} is the dose obtained by the simulation, and D_{max_sim} is the maximum dose deposition in the phantom as calculated in the MC simulation.

The delta index is calculated for each voxel of the in-field area of the beam inside the scoring volume. Then a passing rate can be calculated from the number of voxels in which the delta index is less than 1% or 3%. However, it was found that the model could predict almost all the voxels with near zero value correctly, which were the majority information. The voxels with the crucial information at the beam center are only less than 1 percent of the voxels in the entire image. Therefore, the passing rate of the delta criterion tended to exceed 99% from the early training iterations. The interpretation of this value only could be misleading.

The Wasserstein distance is implemented as part of the training process for Wasserstein GANs (WGANs) to guide the generator towards producing samples that match the real data distribution. However, calculating the Wasserstein distance directly involves finding the optimal transport plan, which can be complex and computationally demanding especially when working with high-dimensional data or large datasets. During hyperparameter tuning, using a computationally less intensive metric helps speed up the process. Moreover, the primary focus on the generalization and performance of the model can be more directly assessed.

In the application to carbon ion dose prediction where pixel-wise similarity or accuracy of dose predictions is a primary concern, the Mean Squared Error (MSE) was chosen to measure the difference between the generated and simulated dose in every voxel, making it easier to interpret in the context of voxel-level tasks.

The MSE is calculated using the following formula:

$$MSE = \frac{1}{N} \sum_{i=1}^N (D_{gen,i} - D_{sim,i})^2 \quad \text{Eq. 13}$$

Where N is the total number of voxels in the dose map, and $D_{gen,i}$ and $D_{sim,i}$ represent the generated dose and (ground truth) simulated dose values in voxel i , respectively.

The MSE loss is monitored during training to ensure that it is decreasing over time. This indicates that the GAN is learning to generate dose distributions that are closer to the ground truth. Evaluation of the model during hyperparameter tuning phase is performed by calculating the MSE on the validation dataset. To evaluate generalization, the testing process includes testing datasets, which are reserved for this purpose and were selected uniformly across all geometries and energies, to ensure that the model performs well across the complete dataset. To ensure that the model generalizes well to new data, the complete set of simulation and density data of the brain geometry of a new patient that is not encountered during the training phase is included in the testing process.

4.5 Code Availability

The complete source code developed in this study, including the model implementation and training scripts, is available on GitHub <https://github.com/pichaya-tap/program> for transparency and reproducibility.

5 Evaluation

5.1 Hyperparameter Tuning

The first experiment focused on the adjustment of learning rates for both the generator and critic to evaluate their impact on the training process. This analysis aimed to recognize how variations in learning rates influence the model's convergence and overall performance. Table 1 and Table 2 display a collection of performance metrics (MSE, average 1% passing

rate and average 3% passing rate) for different learning rates. Figure 16 presents the outcomes from the experiment, specifically comparing the MSE achieved during the model's training. It is important to note that the reported MSE values are based on data normalization within the range of 0-1. While examining the results of the selected trials depicted in the figure, a consistent decreasing trend in MSE over training epochs is evident. Notably, setting the learning rate for the generator (LRG) to 0.003 and for the critic (LRC) to 8E-06 resulted in a more rapid decrease of MSE with low fluctuation. While a learning rate of 3E-05 yielded a slightly lower average MSE, it introduced more fluctuation between epochs. In contrast, a learning rate of 8E-06 offered both stability and faster convergence of MSE in the training process. Considering these observations, 8E-06 was chosen as the preferred learning rate.

Table 1 Performance metrics at varying critic learning rates.

Generator Learning Rate	Performance Metric	Critic Learning Rate			
		0.000006	0.000008	0.00001	0.00003
0.003	MSE [10^{-5}]	1.75 $\pm$ 0.44	1.53 $\pm$ 0.26	2.30 $\pm$ 0.66	1.47 $\pm$ 0.32
	1% passing rate [%]	99.48 $\pm$ 0.06	99.48 $\pm$ 0.06	99.35 $\pm$ 0.07	99.48 $\pm$ 0.06
	3% passing rate [%]	99.80 $\pm$ 0.03	99.81 $\pm$ 0.03	99.73 $\pm$ 0.04	99.80 $\pm$ 0.03

Table 2 Performance metrics at varying generator learning rates.

Critic Learning Rate	Performance Metric	Generator Learning Rate			
		0.0009	0.001	0.003	0.005
0.000008	MSE [10^{-5}]	2.10 $\pm$ 0.37	1.62 $\pm$ 0.36	1.53 $\pm$ 0.26	2.12 $\pm$ 0.44
	1% passing rate [%]	99.81 $\pm$ 0.02	99.82 $\pm$ 0.01	99.48 $\pm$ 0.06	99.82 $\pm$ 0.017
	3% passing rate [%]	99.93 $\pm$ 0.01	99.94 $\pm$ 0.01	99.81 $\pm$ 0.03	99.93 $\pm$ 0.01

Upon experimenting with various hyperparameters according to section [4.3 Model Training](#), it was observed that when setting the critic iteration to 5 and lambda (a hyperparameter that controls the strength of the gradient penalty term) to 10, the model exhibited the most stable training and demonstrated good performance. Two widely employed optimizers, Adam and RMSprop, were also investigated as shown in Figure 17. Over a training span of

200 epochs, RMSprop exhibited a marginally superior performance in minimizing MSE compared to Adam. This subtle difference suggested that, within the specified training duration, RMSprop might offer a slight advantage in achieving lower MSE values. Remarkably, these configurations align with the general recommendations from the original WGAN paper (Arjovsky et al., 2017). The RMSProp optimizer was recommended by (Arjovsky et al., 2017) rather than momentum-based optimizers such as Adam because it could offer more stability in the model training and could perform better. The default settings contributed to stable training and favorable performance outcomes.

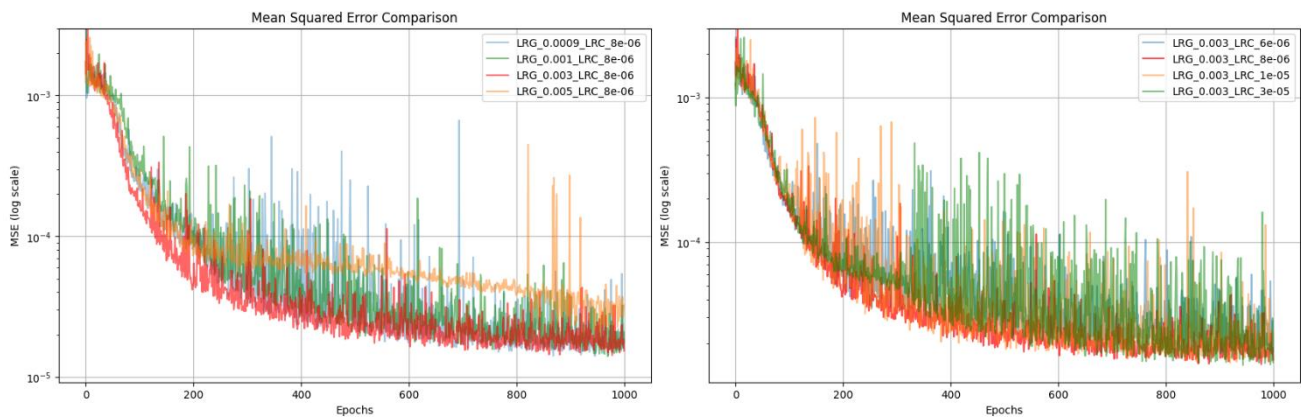


Figure 16 Performance comparison between different learning rates: varying generator learning rates (left) and varying critic learning rates (right)

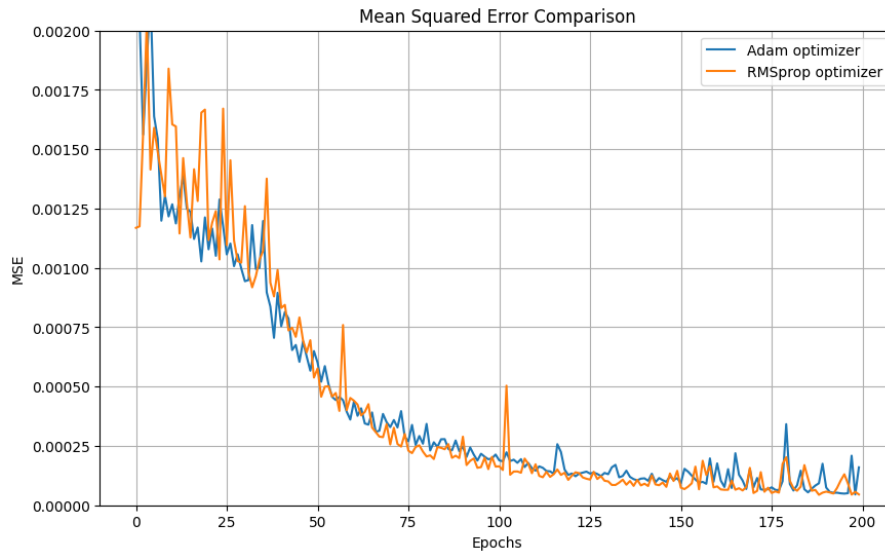
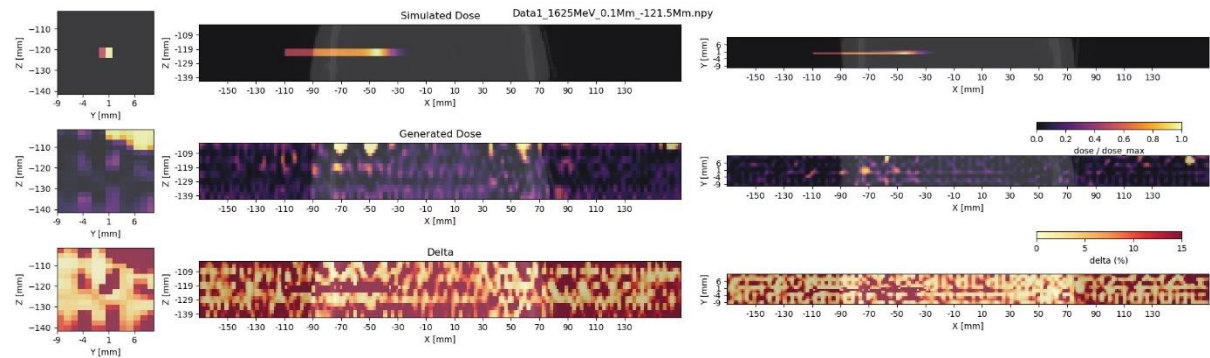


Figure 17 Comparison between Adam and RMSprop optimizers.

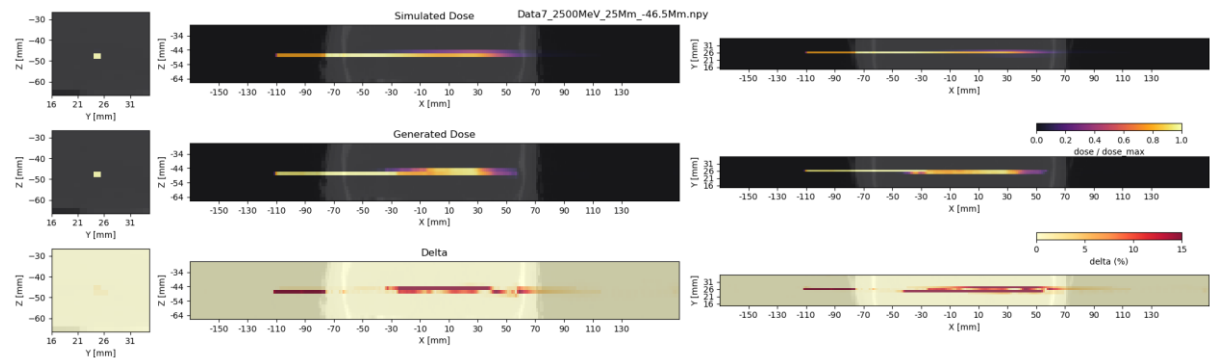
5.2 Performance

From the previously identified optimal hyperparameters, random samples generated by the GAN model were observed during both training and validation phases and are visually presented in Figure 18. The figure showcases samples in the form of heat maps representing the simulated dose, generated dose, and their delta in three orthogonal planes. Within the first initial hundred training epochs, the model demonstrates an improved ability to predict dose distributions that closely align with the ground truth. This suggests a strong learning capability of the model based on the input data. Notably, beyond five hundred training epochs, the model consistently produces dose prediction results that are deemed satisfactory.

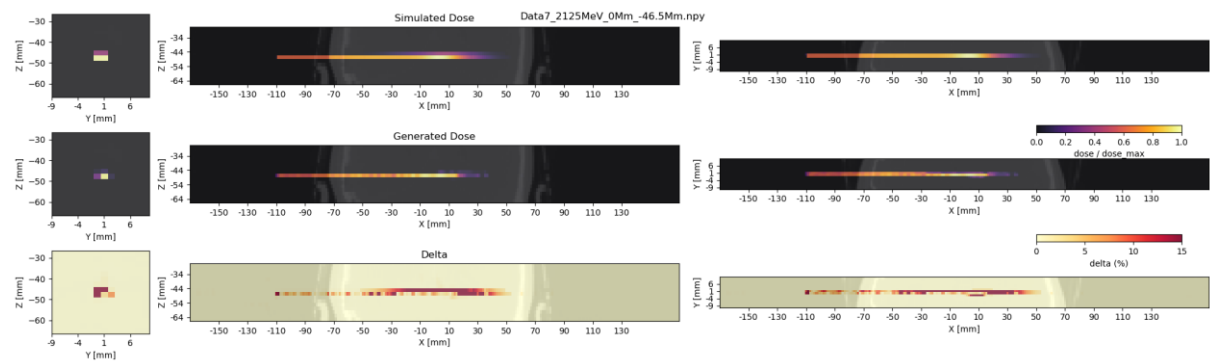
a) Training epoch = 1



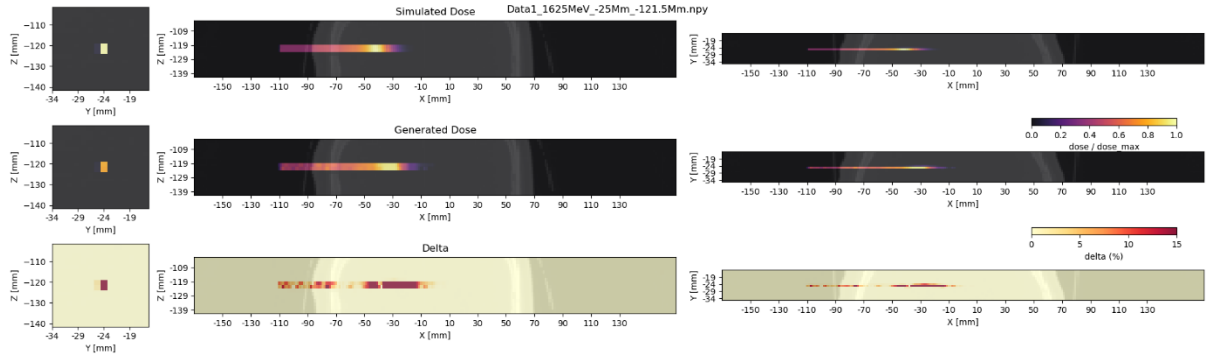
b) Training epoch = 50



c) Training epoch = 100



d) Training epoch = 150



e) Training epoch = 200

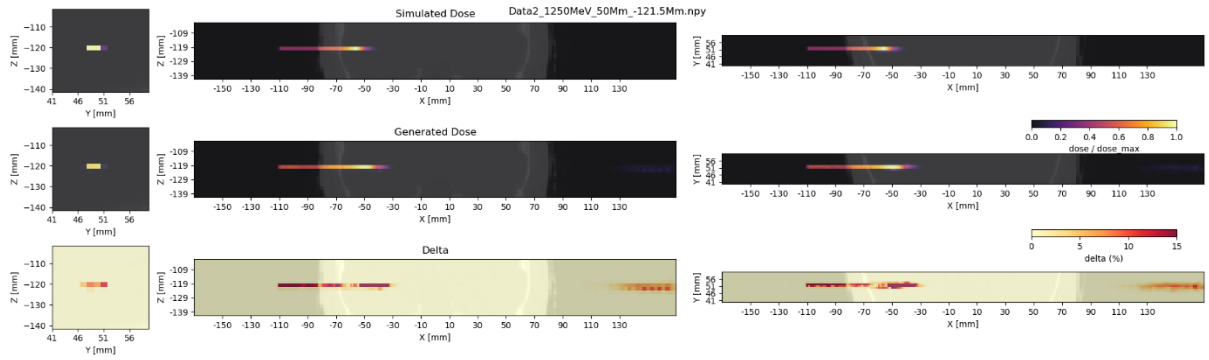


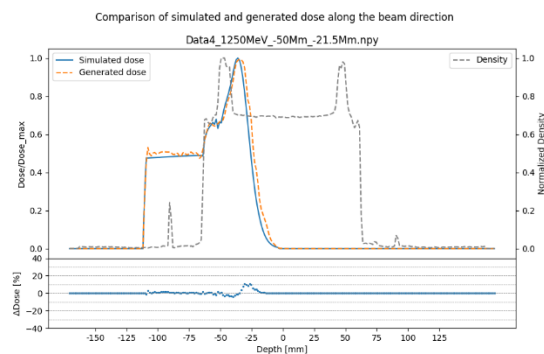
Figure 18 Heat maps representing sample simulated dose distribution (top), generated dose distribution (middle), and their delta (bottom) in three orthogonal planes: YZ plane (left), XZ plane (middle), and XY plane (right). X axis corresponds to beam direction. YZ plane was plotted at the x-value of maximum dose, indicating the probable location of the Bragg peak.

With the optimal hyperparameters, the model was retrained and then the performance of the model was analyzed with the test dataset and unseen geometry. After 1000 training iterations, the GAN model demonstrated a high proficiency in predicting the dose distribution. Table 3 presents performance metrics for the model evaluated on both the validation and test datasets as well as the unseen geometry. These metrics provide insights into the model's performance, showcasing achievement in MSE and passing rates when tested with the test dataset. These observations reveal the model's effectiveness in capturing and generalizing patterns from the data. However, the model faces challenges with the unseen geometry as conditional input, leading to a significantly higher MSE. Further discussion on this issue will be presented in the next section.

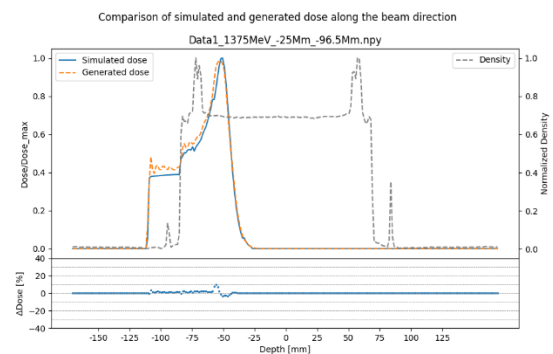
Table 3 Assessing model performance on validation and test datasets.

Performance Metric	Validation Dataset	Test Dataset	Unseen Geometry
MSE [10^{-5}]	1.75 ± 0.44	2.06 ± 0.31	106.90 ± 9.76
1% passing rate [%]	99.48 ± 0.06	99.80 ± 0.02	99.59 ± 0.03
3% passing rate [%]	99.80 ± 0.03	99.93 ± 0.01	99.71 ± 0.02

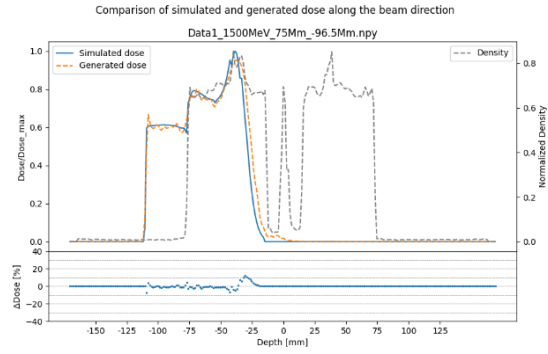
Figure 19 presents random samples of prediction outcomes from the model using the test dataset. Comparison between the normalized MC simulated dose and the generated dose from the GAN model is shown for various carbon beam energy values ranging from 1250 MeV to 3000 MeV. The plot shows dose distribution at the center of the beam in the beam direction, revealing a distinct Bragg peak within the head region. The relative deviations between simulated and generated dose distribution of the test data are mostly within 10% and the model shows a good ability to interpolate to unseen test dataset. In cases where there is more occurrence of density transition regions, the model exhibits poor performance, with deviations exceeding 10% (f). It is noted that the deviation is more noticeable in high-energy scenarios than in low-energy scenarios, particularly when the Bragg peak extends deeper into the head region (m).



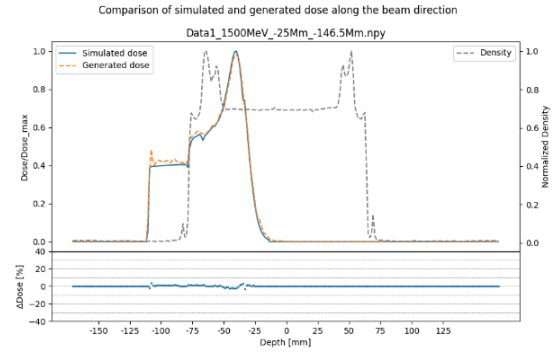
(a) 1250 MeV



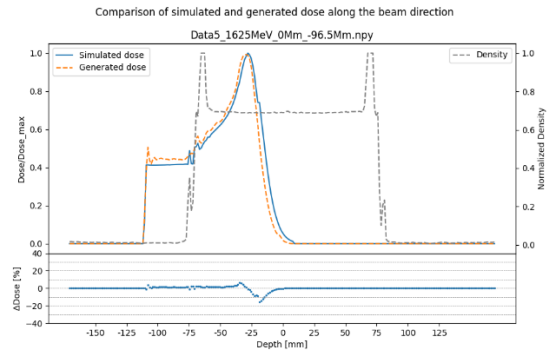
(b) 1375 MeV



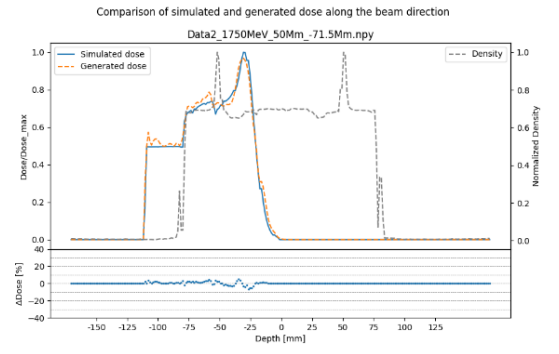
(c) 1500 MeV



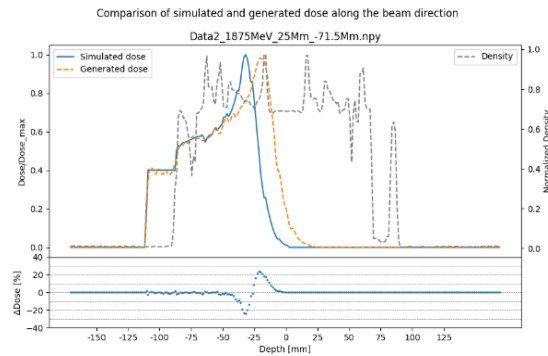
(c) 1500 MeV



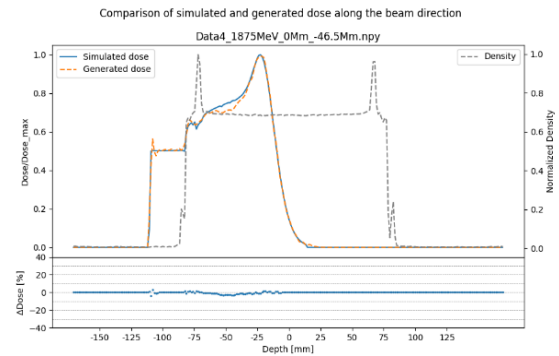
(d) 1625 MeV



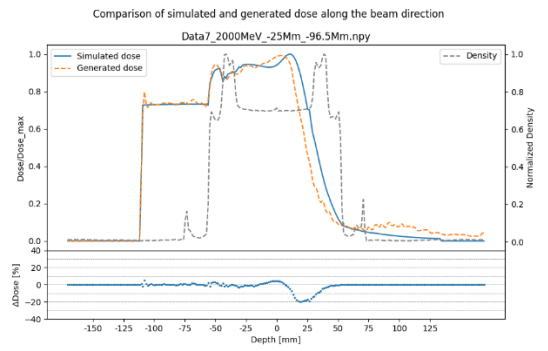
(e) 1750 MeV



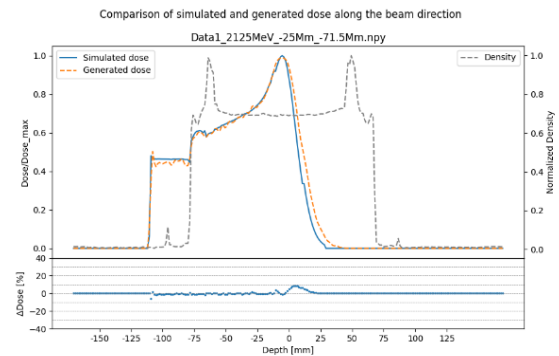
(f) 1875 MeV



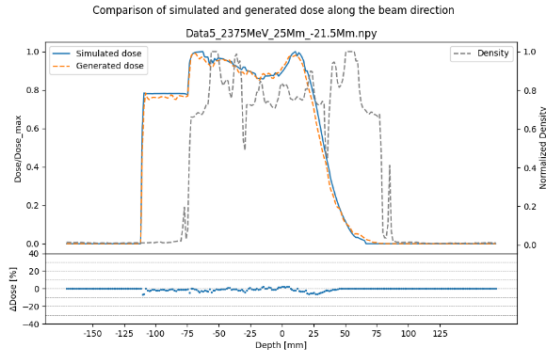
(g) 1875 MeV



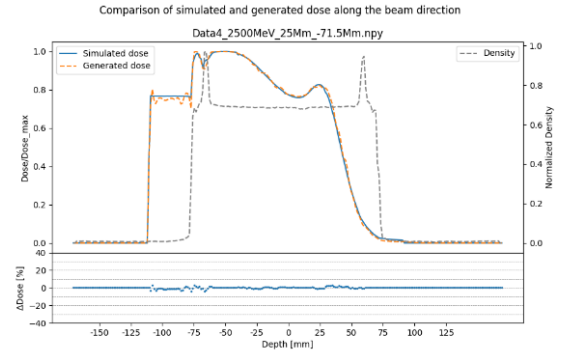
(h) 2000 MeV



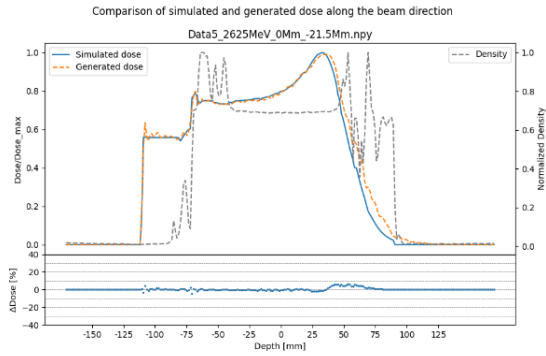
(i) 2125 MeV



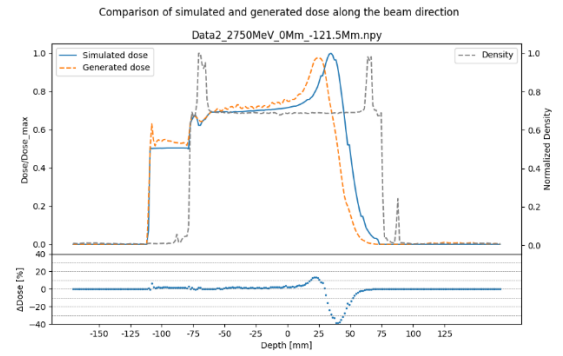
(j) 2375 MeV



(k) 2500 MeV



(l) 2625 MeV



(m) 2750 MeV

Figure 19 Samples of prediction outcomes from the best model using the test dataset. Comparison between the MC simulated dose (blue) and the generated dose from GAN model (orange) is shown for different beam energies.

To explore the generalization capability of the trained model, the performance on a held-out test dataset is shown in Figure 20. It displays the MSE of the predicted dose distribution for all source positions in the head (circles), overlayed to the geometry slice image. The color-coding illustrates the range of MSE values. The highest MSE values are found when the beam traverses the skull or propagates close to it, while the lowest values are found when the beam is located in the inner part of the brain. However, exceptions can also be seen such as on the right top corner of the beam array.

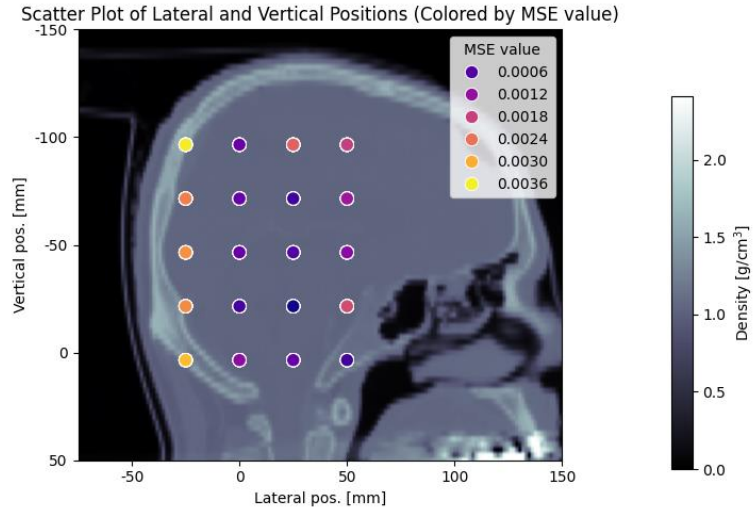


Figure 20 Average MSE at different source positions when tested with the test dataset.

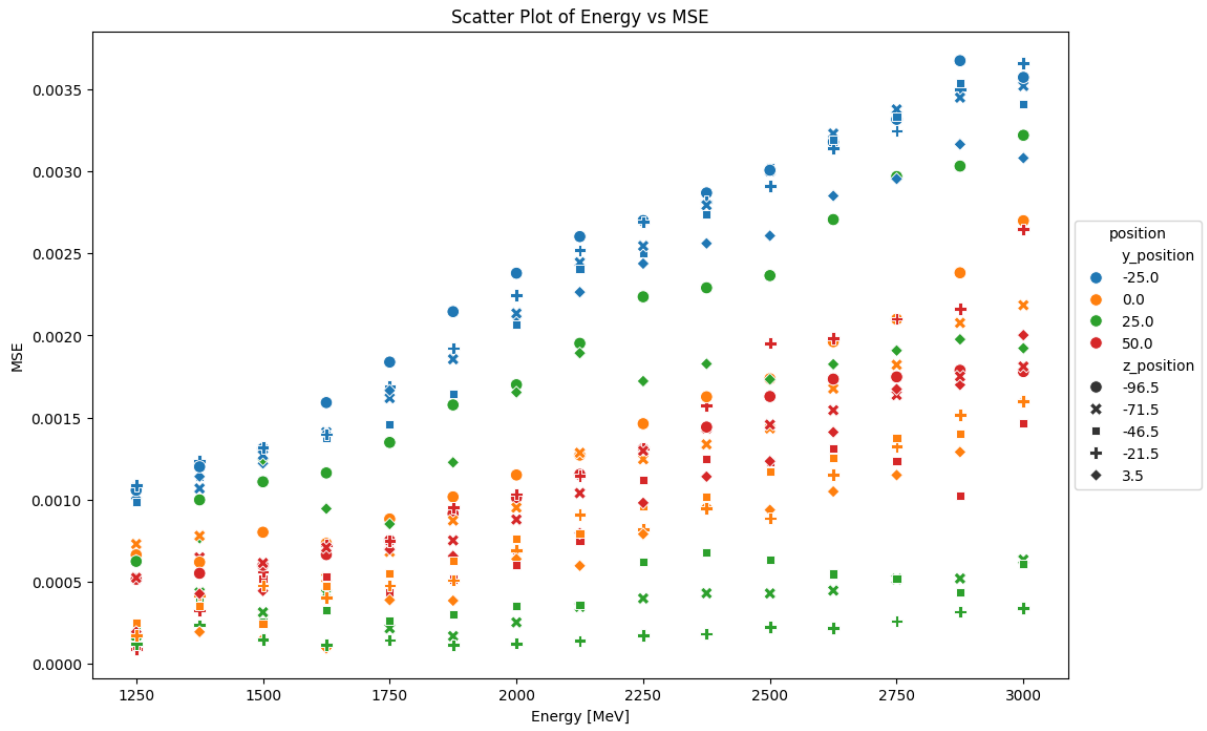


Figure 21 Average MSE trends across energy levels and beam positions. Lateral positions are color-coded and vertical positions are symbol-coded.

When plotting the MSE on the y-axis for beam energies ranging from 1250 MeV to 3000 MeV on the x-axis, an observable trend emerges, suggesting a dependence on the beam energy. The MSE values are also influenced by the lateral (y_{position}) and vertical

(z_position) positions of the beam source. In Figure 21, the lateral position is color-coded, and the vertical position is symbol coded.

Lastly, the time required for one dose prediction using Geant4 ranged from 1 to 3 hours, depending on the beam energy and geometry factors. For the trained WGAN model, the average computational time per prediction was 0.012 seconds on a CPU and 0.009 seconds on a GPU.

Table 4 Time per prediction by GAN model

Model	Time per prediction	Relative speed
Geant4	~ 1 - 3 hours	1
GAN (1 CPU)	0.0116 s	~ 6×10^5
GAN (1 GPU)	0.0088 s	~ 8×10^5

5.3 Discussion

The GAN model showed promising performance for rapid dose calculation through machine learning. Dose prediction using the trained GAN model was five orders of magnitude faster than a Monte Carlo simulation with Geant4, reducing the prediction time from 1 hour to a few microseconds. The model was successful in predicting dose that closely matches the ground truth in some cases. Notably, it achieved a high proficiency in predicting dose distribution on the test dataset, with an average passing rate of over 99.6% in all cases and an average MSE per voxel of $(2.06 \pm 0.31) \times 10^{-5}$.

While (Mentzel et al., 2022) reported 1% and 3% passing rates of 96.3% and 99.6% respectively with broad beam, the unique characteristics of carbon ion beam introduce variations in the beam profile. This makes a direct comparison of the results less straightforward. Nevertheless, with this information as a benchmark, the developed GAN model performed comparably to the existing methodology by (Mentzel et al., 2022). The

results demonstrate that the developed GAN model achieved fast and accurate dose prediction for carbon ion beam.

However, it is crucial to acknowledge that the model encountered challenges when presented with unseen geometries as its conditional input, resulting in the MSE being two orders of magnitude higher. This discrepancy arose when dealing with the unseen geometry of an individual patient, as opposed to the test dataset, which comprised five patients encountered during the training phase. To address this, examining the characteristics of the unseen geometry to identify what sets it apart from the other test datasets would be beneficial but was beyond the scope of this work.

Considering additional performance metrics will be essential to enhance the understanding of the model's performance on the unseen geometry. In voxel-wise MSE calculations, minor errors of around 1 mm corresponding to a single voxel within the generated dose prediction can significantly affect the overall MSE result, making it appear worse than it truly is. For instance, even if the overall dose distribution appears similar, a slight shift of one voxel due to this error may lead to a disproportionately high MSE. In such cases, one can in future also consider other performance metrics, like the Fréchet Inception Distance (FID), for a more accurate evaluation, especially when small errors in individual voxels have a substantial influence on the overall assessment.

The identified dependencies of MSE on beam energies and source positions suggested promising avenues for future exploration. Examining the underlying mechanisms and dependencies could provide valuable insights for the fine-tuning of the model and improving its accuracy in dose prediction. Moreover, obtaining more datasets from additional patients or different geometries for training and testing the model could offer further valuable insights in future studies. The training process could benefit from employing techniques such as leave-one-patient-out cross-validation to enhance the robustness and generalization capabilities of the model.

6 Summary

This master thesis served as a proof-of-concept exploration into advancing treatment planning in carbon ion radiation therapy through the development and application of a Wasserstein Generative Adversarial Network (WGAN). The core theme revolves around integrating artificial intelligence to accelerate dose distribution calculations. A conditional WGAN was developed with U-Net architecture of the generator and a convolutional network encoding the Wasserstein distance. The conditional inputs were density matrices and simulated dose maps from Geant4 simulations of carbon ion beams in water. The training of the network utilized simulated dose maps of carbon ion beams in the human brain based on data taken from the GLIS-RT data set.

The training process involved random splitting of the dataset into training, validation, and test sets, enabling robust model evaluation. The Wasserstein loss function was employed during training, with the mean-squared error (MSE) serving as the evaluation metrics. GPU acceleration was utilized to expedite convergence and leverage parallel processing capabilities. The network was trained using a meticulous hyperparameter tuning strategy, exploring a range of values for generator learning rate, critic learning rate, gradient penalty weight λ , number of critic iterations, and optimizer. The optimizer RMSprop with a generator learning rate of 0.003, a critic learning rate of $8E-06$, a λ of 10, and 5 critic iterations showed the best performance for convergence on the validation set.

To ensure effective generalization, the model underwent retraining and the model's performance was further assessed on the test data and unseen geometry. The GAN model demonstrated exceptional efficiency, achieving a five-orders-of-magnitude reduction in dose calculation time compared to Geant4 simulations. It successfully predicted dose distributions with high accuracy on the test dataset, producing greater than 99% passing rate in all cases. However, challenges arose when the model encountered unseen geometries. The model performed worst when the beam traversed or was close to the skull. Further refinement will be essential to address these challenges and enhance the model's adaptability to diverse geometric scenarios. The limitation observed with unseen

geometries compared to the test dataset indicates the necessity of including more diverse geometries in the training process. Despite the relatively limited availability of geometries and datasets compared to reality, this study stands as a proof-of-concept for future research related to machine learning for rapid dose calculation in carbon ion radiation therapy.

Appendix

A. Dataset: MC Simulation of a therapeutic carbon ion beam in head

This Dataset was generated in May 2023 by Miriam Schwarze

The code can be found in: <https://gitlab1.ptb.de/MiriamSchwarze/simudosemapheavyions>

GENERAL INFORMATION

1. Title of dataset: MC Simulation of a therapeutic carbon ion beam in head
2. Author/ Creator information
 - A. Miriam Schwarze, PTB, miriam.schwarze@ptb.de
 - B. Leo Thomas, PTB, leo.thomas@ptb.de
 - C. Hans Rabus, PTB, hans.rabus@ptb.de
3. Project: Generation of a dataset for the AI based dosemap prediction
4. Date of data collection: May 2023
5. CT Head images from Cancer Imaging Archive:
<https://wiki.cancerimagingarchive.net/pages/viewpage.action?pageId=95224486>

ABSTRACT

The aim of the project is to develop a Neural Network for heavy ion dose and LET prediction. Based on CT images a Monte-Carlo radiation transport simulation will be performed using Geant4 to calculate dose maps which serve as a training dataset. Then two master students will test different network architectures for dosemap prediction.

METHODOLOGICAL INFORMATION

1. Geant4 version Name: geant4-11-00-patch-02 [MT] (25-May-2022)

<< in Multi-threaded mode >>

Copyright : Geant4 Collaboration

References : NIM A 506 (2003), 250-303

: IEEE-TNS 53 (2006), 270-278

: NIM A 835 (2016), 186-225

WWW : <http://geant4.org/>

2. This is the raw data obtained from the simulations (details about the simulation can be found in the README.md file in gitlab)

DATA & FILE OVERVIEW

1. CASES

Energy = [1250, 1500, 1750, 2000, 2250, 2500, 2750, 3000]

Beam Position X = [-100, 0]

Beam Position Y = [-100, -75, -50, -25, 0, 25, 50, 75, 100]

2. FILE NAME CONVENTION

Dosemaps named like:

{ENERGY}MeV_{ROTATIONX}Deg_{ROTATIONY}Deg_{ROTATIONZ}Deg_{POSITIONX}Mm_{POSITIONY}Mm.out

with ENERGY = Energy of primary carbon ion

ROTATION = Rotation of the head (until now not applied)

POSITION = Position of the beam = Translation of the head

3. Units of measurement:

Energies: MeV

Lengths: mm

Dose: Gy

B. MC Simulation Configuration

Parameters	Dataset1	Dataset2	Dataset4	Dataset5	Dataset7	Dataset8
Voxels (X)	256	256	256	256	256	256
Voxels (Y)	256	256	256	256	256	256
Voxels (Z)	126	159	125	131	136	136
Extension in X (mm)	-181 to 181	-194.5 to 194.5	-170.5 to 170.5	-176.5 to 176.5	-175 to 175	-186 to 186
Extension in Y (mm)	-181 to 181	-194.5 to 194.5	-170.5 to 170.5	-176.5 to 176.5	-175 to 175	-186 to 186
Extension in Z (mm)	-229 to 86	-271 to 126.5	-188.75 to 123.75	-208.25 to 119.25	-199.25 to 140.75	-186.25 to 153.75
fVoxelHalfDimX (mm)	0.707031	0.759766	0.666016	0.689453	0.683594	0.726562
fVoxelHalfDimY (mm)	0.707031	0.759766	0.666016	0.689453	0.683594	0.726562
fVoxelHalfDimZ (mm)	1.25	1.25	1.25	1.25	1.25	1.25
Total Pixels	8257536	10420224	8192000	8585216	8912896	8912896
Placing voxel container volume at position (mm)	(0,0,-71.5)	(0,0,-72.25)	(0,0,-32.5)	(0,0,-44.5)	(0,0,-29.25)	(0,0,-16.25)
Beam start position X (mm)	-110	-110	-110	-110	-110	-110
Beam start positions Y (mm)	-50, -25, 0, 25, 50, 75	-25, 0, 25, 50, 75	-50, -25, 0, 25, 50, 75	-50, -25, 0, 25, 50, 75	-25, 0, 25, 50, 75	-25, 0, 25, 50
Beam start positions Z (mm)	-146.5, -121.5, -96.5, -71.5, -46.5	-171.5, -146.5, -121.5, -96.5, -71.5	-71.5, -46.5, -21.5, 3.5	-96.5, -71.5, -46.5, -21.5	-96.5, -71.5, -46.5, -21.5	-96.5, -71.5, -46.5, -21.5, 3.5
Beam direction	(1,0,0)	(1,0,0)	(1,0,0)	(1,0,0)	(1,0,0)	(1,0,0)
Number of Simulations	450	375	360	360	300	300

C. Dose Map Construction

The dose data are obtained from MC simulations as a linear array of voxel indices and the corresponding dose values. This section describes the code used to process the dose data obtained from MC simulations and store it in a more structured form, specifically within a three-dimensional voxel grid. It converts dose data from a linear index format to a three-dimensional voxel grid representation. This transformation is crucial for further analysis and visualization of carbon ion simulation results. The voxel grid is defined by the constants `VOXEL_NUMBER_X`, `VOXEL_NUMBER_Y`, and `VOXEL_NUMBER_Z`, representing the dimensions of the grid in the X, Y, and Z axes, respectively.

The conversion process is as following:

- For each eligible file, the code initiates the conversion process.
- It reads the dose data from the file using pandas, assuming a tab-separated format.
- An index array is created to represent all possible combinations of indices in the voxel grid.
- The dose data is filtered to include only the indices within the voxel grid.
- A three-dimensional dose distribution array is initialized with zeros.
- The X, Y, and Z indices are calculated from the filtered data and the corresponding dose values are used to populate the dose map array.
- The resulting dose map is saved as a NumPy array.

The code demonstrates a systematic approach to handling and processing simulation data for further usage.

```

# Shape of voxel
VOXELNUMBER_X = 256
VOXELNUMBER_Y = 256
VOXELNUMBER_Z = 126

input_filename = '/gpfs001/scratch/schwar14/simudosemapheavyions/DosemapInBrain/build_folders/Miriam/Data1_1250MeV_-50,Mm_-146.5,Mm.out'

dicom_data = pd.read_table(input_filename, names=['Index', 'Dose'], sep=' ', engine='python')

# Create an index array from 0 to VOXELNUMBER_X * VOXELNUMBER_Y * VOXELNUMBER_Z
index_array = np.arange(VOXELNUMBER_X * VOXELNUMBER_Y * VOXELNUMBER_Z)

# Filter the dicom_data DataFrame for matching indices
filtered_data = dicom_data[dicom_data['Index'].isin(index_array)]

# Create an empty dosemap array
dosemap = np.zeros(shape=(VOXELNUMBER_Z, VOXELNUMBER_Y, VOXELNUMBER_X))

# Calculate the x, y, z indices from the filtered data
y_indices = filtered_data['Index'] // (VOXELNUMBER_Z * VOXELNUMBER_Y)
x_indices = (filtered_data['Index'] // VOXELNUMBER_Z) % VOXELNUMBER_Y
z_indices = filtered_data['Index'] % VOXELNUMBER_Z

# Assign the dose values to the corresponding positions in the dosemap array
dosemap[z_indices, y_indices, x_indices] = filtered_data['Dose'].values

```

Figure 22 Python script used for conversion of the simulation data to a 3D dose map

List of symbols and acronyms

symbol	unit	description
ε	J or eV	energy
D	Gray	dose
m	kg	mass
ρ	kg / m ³	density
e	Coulomb	elementary charge
z	-	noise input
p_z	-	data distribution over the noise input
x	-	data
p_r	-	data distribution over real samples x
$\hat{x}$	-	interpolation between real and generated samples
$D(x)$	-	output of the discriminator
p_g	-	generator's distribution over data x
$G(z)$	-	output of the generator when given a random noise vector z
W	-	Wasserstein distance
c	-	threshold of critic's weight
L_c	-	critic loss
L_g	-	generator loss
λ	-	weight of gradient penalty
δ	-	delta index
N	-	total number of voxels

Acronym	description
LRG	learning rate for the generator
LRC	learning rate for the critic

List of figures

Figure 1 Statistics of patients treated in particle therapy facilities worldwide.(Jermann, 2016).....	8
Figure 2 LET and RBE of carbon-ion beams (290MeV, SOBP=60mm).(Tsuji, 2017).....	9
Figure 3 Architecture of a generative adversarial network. (Image source: KDNuggets).	11
Figure 4 Pseudocode of WGAN with gradient penalty (Gulrajani et al., 2017)	19
Figure 5 Steps in the ML model development process.....	21
Figure 6 nvidia-smi tools on the node showing GPU information (top). Detailed information about the compute node, including its hardware specifications (bottom).	22
Figure 7 Profiles of the calculated dose distribution of carbon ion in the head along three directions.	24
Figure 8 Location of the beam in the lateral and vertical position combinations used in the MC simulations (left). Beam in x-direction from left to right of the head shown as red dash lines (middle, right).....	26
Figure 9 Profiles of the calculated dose distribution of carbon ion in water at 2000 MeV along three directions.	27
Figure 10 The simulated depth-dose profiles of carbon ion beams in water phantom at different beam energies.....	28
Figure 11 Samples from different datasets compared with their respective doses in the water.	33
Figure 12 The generator model architecture.....	34
Figure 13 Generator model summary.....	35
Figure 14 Critic model architecture.	36
Figure 15 Critic model summary	37
Figure 16 Performance comparison between different learning rates: varying generator learning rates (left) and varying critic learning rates (right).....	42
Figure 17 Comparison between Adam and RMSprop optimizers.	43
Figure 18 Heat maps representing sample simulated dose distribution (top), generated dose distribution (middle), and their delta (bottom) in three orthogonal planes: YZ plane (left), XZ plane (middle), and XY plane (right). X axis corresponds to beam direction. YZ plane was plotted at the x-value of maximum dose, indicating the probable location of the Bragg peak.....	45
Figure 19 Samples of prediction outcomes from the best model using the test dataset. Comparison between the MC simulated dose (blue) and the generated dose from GAN model (orange) is shown for different beam energies.....	48
Figure 20 Average MSE at different source positions when tested with the test dataset.	49
Figure 21 Average MSE trends across energy levels and beam positions. Lateral positions are color-coded and vertical positions are symbol-coded.	49
Figure 22 Python script used for conversion of the simulation data to a 3D dose map.....	57

List of tables

Table 1 Performance metrics at varying critic learning rates.....	41
Table 2 Performance metrics at varying generator learning rates.....	41
Table 3 Assessing model performance on validation and test datasets.....	46
Table 4 Time per prediction by GAN model.....	50

Bibliography

AAPM Report 85. (2004). *Tissue inhomogeneity corrections for megavoltage photon beams*.

Radiation Therapy Committee Task Group 65. Medical Physics Publishing, Madison, WI.

https://www.aapm.org/pubs/reports/rpt_85.pdf

Agostinelli, S., Allison, J., Amako, K., Apostolakis, J., Araujo, H., Arce, P., Asai, M., Axen, D., Banerjee,

S., Barrand, G., Behner, F., Bellagamba, L., Boudreau, J., Broglia, L., Brunengo, A., Burkhardt,

H., Chauvie, S., Chuma, J., Chytrcek, R., ... Zschesche, D. (2003). Geant4—A simulation

toolkit. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators,*

Spectrometers, Detectors and Associated Equipment, 506(3), 250–303.

[https://doi.org/10.1016/S0168-9002\(03\)01368-8](https://doi.org/10.1016/S0168-9002(03)01368-8)

Andreo, P., Burns, D. T., Nahum, A. E., Seuntjens, J., & Attix, F. H. (2017). *Fundamentals of ionizing*

radiation dosimetry. Textbook / Pedro Andreo, David T. Burns, Alan E. Nahum, Jan Seuntjens,

and Frank H. Attix. Wiley-VCH.

Arjovsky, M., & Bottou, L. (2017). *Towards Principled Methods for Training Generative Adversarial*

Networks (arXiv:1701.04862). arXiv. <http://arxiv.org/abs/1701.04862>

Arjovsky, M., Chintala, S., & Bottou, L. (2017). *Wasserstein GAN* (arXiv:1701.07875). arXiv.

<http://arxiv.org/abs/1701.07875>

Barragán-Montero, A. M., Thomas, M., Defraene, G., Michiels, S., Haustermans, K., Lee, J. A., &

Sterpin, E. (2021). Deep learning dose prediction for IMRT of esophageal cancer: The effect

- of data quality and quantity on model performance. *Physica Medica*, 83, 52–63.
<https://doi.org/10.1016/j.ejmp.2021.02.026>
- Bert, J., Perez-Ponce, H., Bitar, Z. E., Jan, S., Boursier, Y., Vintache, D., Bonissent, A., Morel, C., Brasse, D., & Visvikis, D. (2013). Geant4-based Monte Carlo simulations on GPU for medical applications. *Physics in Medicine and Biology*, 58(16), 5593–5611.
<https://doi.org/10.1088/0031-9155/58/16/5593>
- Douglass, M. J. J. (2020). Book Review: Hands-on Machine Learning with Scikit-Learn, Keras, and Tensorflow, 2nd edition by Aurélien Géron: O’ Reilly Media, 2019, 600 pp., ISBN: 978-1-492-03264-9. *Physical and Engineering Sciences in Medicine*, 43(3), 1135–1136.
<https://doi.org/10.1007/s13246-020-00913-z>
- Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative Adversarial Networks* (arXiv:1406.2661). arXiv.
<http://arxiv.org/abs/1406.2661>
- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). *Improved Training of Wasserstein GANs* (arXiv:1704.00028). arXiv. <http://arxiv.org/abs/1704.00028>
- Jermann, M. (2016). *PTCOG*. <https://www.ptcog.site/index.php/patient-statistics-2>
- Kearney, V., Chan, J. W., Wang, T., Perry, A., Descovich, M., Morin, O., Yom, S. S., & Solberg, T. D. (2020). DoseGAN: A generative adversarial network for synthetic dose prediction using attention-gated discrimination and generation. *Scientific Reports*, 10(1), 11073.
<https://doi.org/10.1038/s41598-020-68062-7>
- Lan, L., You, L., Zhang, Z., Fan, Z., Zhao, W., Zeng, N., Chen, Y., & Zhou, X. (2020). Generative Adversarial Networks and Its Applications in Biomedical Informatics. *Frontiers in Public Health*, 8, 164. <https://doi.org/10.3389/fpubh.2020.00164>
- Mentzel, F., Kröninger, K., Lerch, M., Nackenhorst, O., Paino, J., Rosenfeld, A., Saraswati, A., Tsoi, A. C., Weingarten, J., Hagenbuchner, M., & Guatelli, S. (2022). Fast and accurate dose predictions for novel radiotherapy treatments in heterogeneous phantoms using conditional

- 3D-UNet generative adversarial networks. *Medical Physics*, 49(5), 3389–3404.
<https://doi.org/10.1002/mp.15555>
- Mirza, M., & Osindero, S. (2014). *Conditional Generative Adversarial Nets* (arXiv:1411.1784). arXiv.
<http://arxiv.org/abs/1411.1784>
- Nguyen, D., Long, T., Jia, X., Lu, W., Gu, X., Iqbal, Z., & Jiang, S. (2019). A feasibility study for predicting optimal radiation therapy dose distributions of prostate cancer patients from patient anatomy using deep learning. *Scientific Reports*, 9(1), 1076.
<https://doi.org/10.1038/s41598-018-37741-x>
- Radford, A., Metz, L., & Chintala, S. (2016). *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks* (arXiv:1511.06434). arXiv.
<http://arxiv.org/abs/1511.06434>
- Russell, S. J., & Norvig, P. (2010). *Artificial Intelligence A Modern Approach* (3rd ed.). Pearson Education, Inc.
- Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., & Chen, X. (2016). *Improved Techniques for Training GANs* (arXiv:1606.03498). arXiv. <http://arxiv.org/abs/1606.03498>
- Sarrut, D., Krah, N., & Létang, J.-M. (2019). *Generative Adversarial Networks (GAN) for compact beam source modelling in Monte Carlo simulations*.
<https://doi.org/10.48550/ARXIV.1907.13324>
- Schwarze, M. (2023). *MC Simulation of a therapeutic carbon ion beam in head*.
<https://gitlab1.ptb.de/MiriamSchwarze/simudosemapheavyions>
- Seco, J., & Verhaegen, F. (Eds.). (2016). *Monte Carlo Techniques in Radiation Therapy* (0 ed.). CRC Press. <https://doi.org/10.1201/b13961>
- Seltzer, S. M., Bartlett, D. T., Burns, D. T., Dietze, G., Menzel, H.-G., Paretzke, H. G., & Wambersie, A. (2011). 5. Dosimetry. *Journal of the ICRU*, 11(1), 23–28. <https://doi.org/10.1093/jicru/ndr003>
- Shanmugamani, R. (2018). *Deep Learning for Computer Vision: Expert techniques to train advanced neural networks using TensorFlow and Keras* (1st ed.). Packt Publishing Ltd.

- Shusharina, N., & Bortfeld, T. (2021). *Glioma Image Segmentation for Radiotherapy: RT targets, barriers to cancer spread, and organs at risk (GLIS-RT)* (Version 1) [dataset]. The Cancer Imaging Archive. <https://doi.org/10.7937/TCIA.T905-ZQ20>
- Tsujii, H. (2017). Overview of Carbon-ion Radiotherapy. *Journal of Physics: Conference Series*, 777, 012032. <https://doi.org/10.1088/1742-6596/777/1/012032>
- Vassiliev, O. N. (2016). *Monte Carlo methods for radiation transport: Fundamentals and advanced topics*. Springer Science+Business Media.
- Weng, L. (2019). *From GAN to WGAN* (arXiv:1904.08994). arXiv. <http://arxiv.org/abs/1904.08994>
- Yoganathan, S. A., & Zhang, R. (2019). An atlas-based method to predict three-dimensional dose distributions for cancer patients who receive radiotherapy. *Physics in Medicine & Biology*, 64(8), 085016. <https://doi.org/10.1088/1361-6560/ab10a0>
- Zhang, Q., Kong, L., Liu, R., & Wang, X. (2021). Ion therapy guideline (Version 2020): Radiation Oncology Physicians Branch of Chinese Medical Doctor Association. *Precision Radiation Oncology*, 5(2), 73–83. <https://doi.org/10.1002/pro6.1120>